Лабораторна робота 2 (Частина 4)

Побудова дифузійної моделі для генерування та відновлення зображень

Мета роботи

Вивчити принципи функціонування генеративних дифузійних моделей, реалізувати прямий процес дифузії шляхом послідовного додавання гауссового шуму до зображень та дослідити вплив його параметрів. Побудувати спрощену нейронну мережу U-Net для прогнозування шуму, навчити модель зворотного дифузійного процесу та застосувати її для генерування нових і відновлення зашумлених зображень. Оцінити якість отриманих результатів та дослідити вплив кількості кроків дифузії й розкладу шуму на точність відновлення і якість генерації.

Теоретичний матеріал

1. Генеративні дифузійні моделі

Генеративне моделювання зображень. Генеративні моделі (Generative Models) – клас моделей машинного навчання, призначених для дослідження закономірностей у навчальних даних і формування нових зразків, які за своїми характеристиками відповідають розподілу вихідної вибірки.

Нехай задано навчальний набір зображень:

$$D = \{x^{(1)}, x^{(2)}, \dots, x^{(N)}\}$$

де $x^{(i)} \in R^{H \times W \times C}$ – окреме зображення; H – його висота; W – ширина; C – кількість каналів; N – кількість зображень у навчальній вибірці.

Припускається, що зображення отримані з невідомого розподілу $p_{\text{data}}(x)$. Завдання генеративної моделі полягає в наближенні цього розподілу параметричною моделлю $p_\theta(x)$:

$$p_\theta(x) \approx p_{\text{data}}(x)$$

де θ – параметри моделі, які визначаються під час навчання.

Після навчання модель можна використовувати для формування нових зображень:

$$x_{\text{new}} \sim p_\theta(x)$$

На відміну від класифікаційних нейронних мереж, які встановлюють належність зображення до певного класу, генеративні моделі намагаються відтворити характерні властивості даних, зокрема форму об’єктів, текстури, просторові залежності та статистичні закономірності.

Дифузійні моделі (Diffusion Models). Дифузійні моделі є класом генеративних моделей, у яких формування даних здійснюється через послідовність стохастичних перетворень. Їхньою основою є два взаємопов’язані процеси (рис.1):

Прямий процес дифузії (Forward Diffusion Process) – поступове додавання випадкового шуму до початкового зображення, унаслідок чого його структура руйнується. Зворотний процес дифузії (Reverse Diffusion Process) – послідовне видалення шуму за допомогою навченої нейронної мережі, що дає змогу отримати зображення із шумового представлення.

У моделі DDPM (Denoising Diffusion Probabilistic Model) прямий процес задається заздалегідь визначеним імовірнісним механізмом, тоді як параметри зворотного процесу визначаються під час навчання нейронної мережі.

На початку прямого процесу використовується реальне зображення x_0. До нього поступово додається шум, і після T кроків отримується зображення x_T, розподіл якого за належно вибраного процесу наближається до стандартного нормального розподілу.

Зворотний процес починається з шумового зображення та послідовно формує менш зашумлені представлення:

$$x_T \to x_{T-1} \to \cdots \to x_1 \to x_0$$

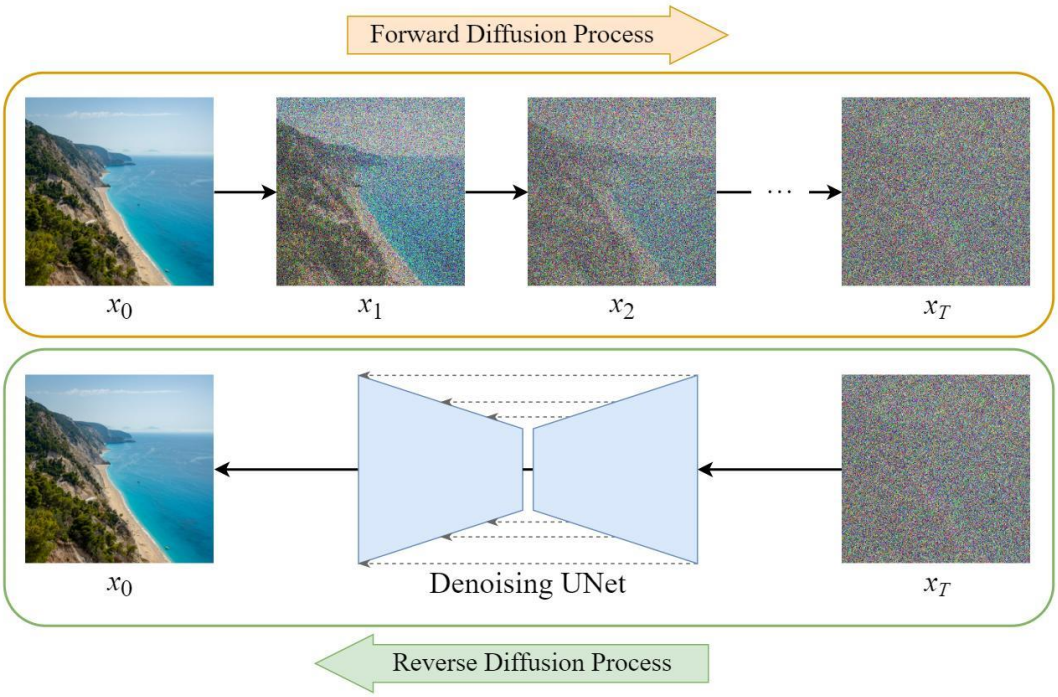


Рис. 1. Загальний принцип роботи дифузійної моделі: пряме зашумлення зображення та зворотне відновлення.

Для виконання такого процесу використовується нейронна мережа, яка навчається прогнозувати шум, присутній у зображенні на заданому кроці дифузії.

Особливість дифузійних моделей полягає в тому, що мережа не генерує повне зображення за один прохід, а бере участь у послідовному уточненні його представлення.

Отже, генерація складного об'єкта розкладається на значну кількість відносно простих операцій, кожна з яких спрямована на зменшення впливу шуму.

Основні застосування дифузійних моделей. Дифузійні моделі використовуються для генерування нових зображень, відновлення пошкоджених даних, підвищення роздільної здатності, заповнення відсутніх фрагментів зображень (inpainting), синтезу медичних зображень та формування штучних навчальних вибірок.

У цій лабораторній роботі розглядається безумовна генерація зображень і відновлення після контрольованого додавання гауссового шуму.

2. Прямий процес дифузії

Прямий процес дифузії (Forward Diffusion Process) – це послідовність перетворень, під час яких до початкового зображення поступово додається гауссовий шум.

Позначимо початкове зображення як x_0. На кожному кроці t, де $t = 1,2,\dots,T$, формується нове зашумлене зображення x_t.

Прямий процес описується марківським ланцюгом:

$$q(x_{1:T} \mid x_0) = \prod_{t=1}^{T} q\left(x_t \mid x_{t-1}\right)$$

Це означає, що розподіл зображення на кроці t залежить лише від зображення на попередньому кроці $t-1$.

Математичний опис одного кроку. У DDPM умовний розподіл зашумленого зображення визначається як:

$$q(x_t \mid x_{t-1}) = \mathcal{N}\left(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t I\right)$$

де x_{t-1} – зображення на попередньому кроці; x_t – зашумлене зображення на поточному кроці; $\beta_t \in (0,1)$ – параметр дисперсії шуму на кроці t; I – одинична матриця; $\mathcal{N}$ – нормальний розподіл.

Відповідне перетворення можна реалізувати за формулою:

$$x_t = \sqrt{1-\beta_t}x_{t-1} + \sqrt{\beta_t}\varepsilon_t$$

де $\varepsilon_t \sim \mathcal{N}(0, I)$ – незалежний випадковий гауссовий шум.

Перший доданок визначає частину сигналу, яка зберігається після зашумлення. Другий доданок відповідає випадковому шуму, доданому на поточному кроці.

Чим більше значення β_t, тим більша інтенсивність шуму, який додається на відповідному кроці.

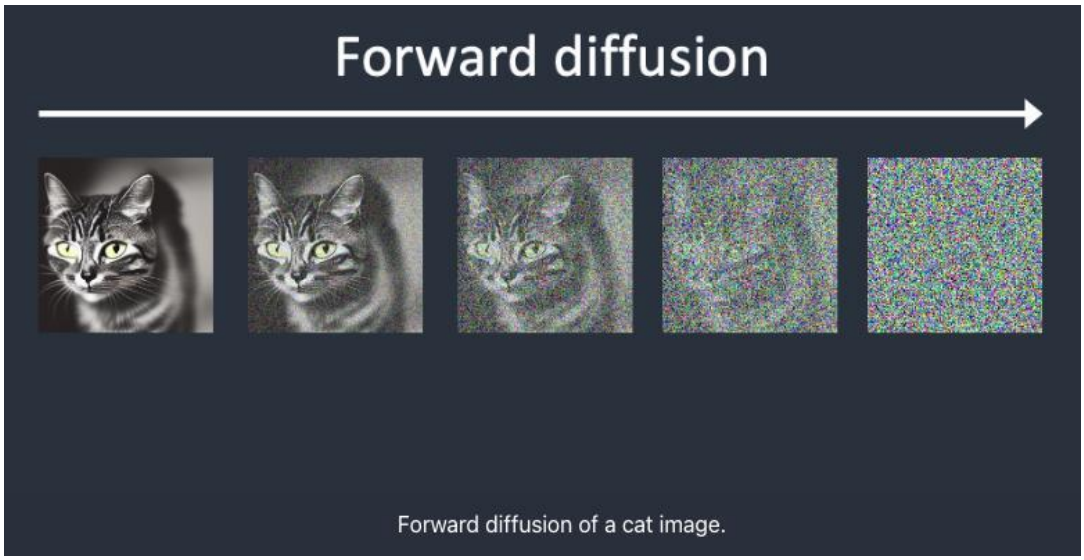


Рис. 2. Схематична послідовність руйнування структури зображення під час прямого процесу дифузії.

На початкових кроках зображення зберігає основні контури об'єкта. Зі збільшенням кількості кроків структура поступово втрачається. Наприкінці достатньо тривалого процесу за відповідного вибору параметрів зашумлене зображення наближається до випадкового гауссового шуму.

Варто зазначити, що кількість кроків сама по собі не гарантує повного руйнування структури зображення. Результат залежить також від величин та сумарного рівня збереження початкового сигналу.

Коефіцієнти шуму. Для спрощення математичних виразів вводять коефіцієнт

$$\alpha_t = 1 - \beta_t$$

Тоді формула одного кроку набуває вигляду:

$$x_t = \sqrt{\alpha_t}x_{t-1} + \sqrt{1-\alpha_t}\varepsilon_t$$

Коефіцієнт α_t характеризує збереження сигналу під час одного кроку дифузії.

Для опису накопиченого впливу всіх попередніх кроків використовують:

$$\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s = 1^t \alpha_s$$

де $\bar{\alpha}_t$ – добуток коефіцієнтів збереження сигналу від першого до поточного кроку.

Зі збільшенням t величина $\bar{\alpha}_t$ зазвичай зменшується, що означає поступове зменшення внеску початкового зображення.

Отримання зашумленого зображення на довільному кроці. Важливою властивістю DDPM є можливість безпосередньо отримати зашумлене зображення x_t із початкового x_0, не виконуючи послідовно всі попередні кроки.

Відповідний умовний розподіл має вигляд:

$$q(x_t \mid x_0) = \mathcal{N}\left(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)I\right)$$

Отже, зашумлене зображення обчислюється за формулою:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\varepsilon$$

де $\varepsilon \sim \mathcal{N}(0, I)$.

Ця формула є однією з основних у DDPM, оскільки використовується безпосередньо під час навчання нейронної мережі.

Вона дає змогу для кожного навчального зображення випадково вибрати крок t і одразу сформувати зашумлену версію, за якою модель навчатиметься прогнозувати доданий шум.

Приклад розрахунку прямого процесу. Нехай значення одного пікселя нормалізованого зображення становить $x_0 = 0.8$.

Припустимо, що для обраного кроку:

$$\bar{\alpha}_t = 0.64, \qquad \varepsilon = 0.5$$

Тоді $x_t = \sqrt{0.64} \cdot 0.8 + \sqrt{1 - 0.64 \cdot 0.05}$. Після підстановки $x_t = 0.94$.

Таким чином, після додавання шуму значення пікселя змінилося з 0.8 до 0.94.

Це приклад одного випадкового значення шуму для одного пікселя. Під час обробки всього зображення шум формується незалежно для кожного пікселя та каналу. Значення зашумлених пікселів можуть виходити за межі діапазону початково нормалізованих зображень – це допустимо в математичній моделі дифузії.

3. Розклад шуму в дифузійних моделях (Noise Schedule)

Розклад шуму (Noise Schedule) – це правило, яке визначає, як змінюється інтенсивність додавання шуму на кожному кроці прямого дифузійного процесу.

У моделі DDPM рівень шуму задається послідовністю параметрів: $\beta_1, \beta_2, \cdots, \beta_T$, де T – загальна кількість кроків дифузії, а β_t – дисперсія шуму, що додається на кроці t.

Вибір розкладу шуму має суттєвий вплив на процес навчання та генерації зображень, оскільки визначає швидкість руйнування початкового сигналу, співвідношення сигналу й шуму на різних кроках та складність задачі відновлення.

У лабораторній роботі розглядаються два способи формування розкладу шуму: лінійний (Linear Noise Schedule) та косинусний (Cosine Noise Schedule).

Лінійний розклад шуму (Linear Noise Schedule). За лінійного розкладу значення β_t рівномірно збільшується від початкового βmin до кінцевого βmax:

$$\beta_t = \beta_{min} + \frac{t-1}{T-1}(\beta_{max} - \beta_{min})$$

де t = 1, …, T, T > 1.

Типовими параметрами для DDPM із T =1000 є:

$$\beta_{\min} = 0.0001, \qquad \beta_{\max} = 0.02$$

Для лабораторної роботи можна використовувати меншу кількість кроків, наприклад T =200, щоб зменшити обчислювальні витрати. Однак під час зміни необхідно контролювати сумарний рівень шуму, оскільки ті самі граничні значення не гарантують однакової інтенсивності зашумлення наприкінці процесу.

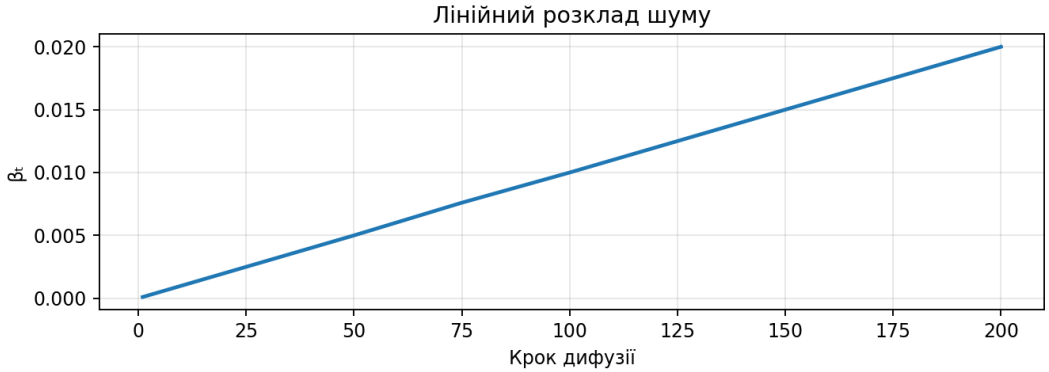


Рис. 3. Зміна дисперсії шуму β_t за лінійного розкладу.

На початкових кроках додається відносно невелика кількість шуму, тому зображення зберігає більшу частину своєї структури. На пізніших кроках дисперсія доданого шуму зростає.

Слід враховувати, що величина β_t описує шум одного кроку, тоді як сумарне зашумлення визначається накопиченим коефіцієнтом $\bar{\alpha}_t$.

Косинусний розклад шуму (Cosine Noise Schedule). Косинусний розклад передбачає нелінійну зміну рівня зашумлення. На відміну від лінійного підходу, він безпосередньо задає характер зміни накопиченого коефіцієнта збереження сигналу $\bar{\alpha}_t$.

Спочатку визначається допоміжна функція:

$$f(t) = \cos^2\left(\frac{\frac{t}{T} + s}{1 + s} \cdot \frac{\pi}{2}\right)$$

де s – невеликий додатний параметр, який регулює форму розкладу. Зазвичай використовують s = 0.008.

Накопичений коефіцієнт визначається як:

$$\bar{\alpha}_t = \frac{f(t)}{f(0)}.$$

Для отримання дискретної послідовності шуму використовують:

$$\beta_t = \min\left(1 - \frac{\bar{\alpha}_t}{\bar{\alpha}_{t-1}}, \beta_{\max}^{\text{clip}}\right)$$

де β_{max}^{clip}, наприклад 0.999, обмежує максимальне значення β_t для забезпечення числової стійкості.

Косинусний розклад дає змогу довше зберігати корисну інформацію в зображенні на початкових і середніх етапах, тоді як наприкінці процесу накопичений рівень шуму суттєво збільшується.

Порівняння лінійного та косинусного розкладів.

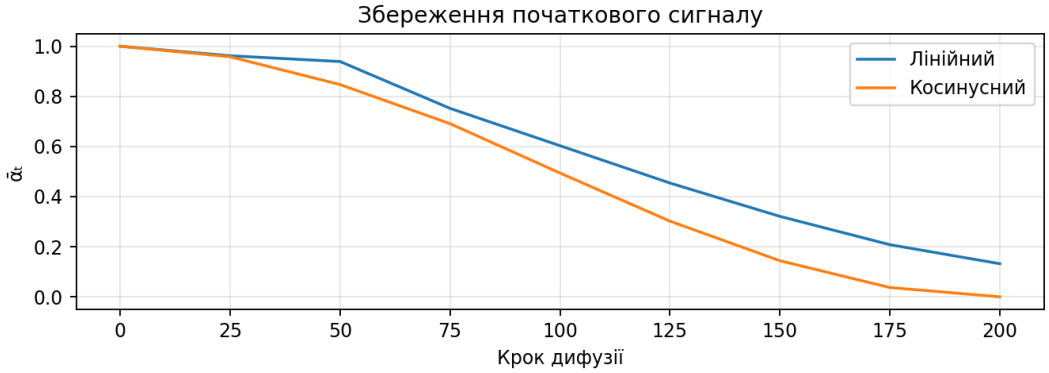


Рис. 4. Ілюстрація зміни накопиченого коефіцієнта $\bar{\alpha}_t$ для двох розкладів. Для однакової кількості кроків кінцеві рівні шуму можуть відрізнятися.

Основні відмінності розкладів наведено в таблиці.

Характеристика	**Linear Schedule**	**Cosine Schedule**
Принцип формування	Лінійне збільшення β_t	Косинусний закон зміни $\bar{\alpha}_t$
Складність реалізації	Низька	Дещо вища
Накопичення шуму	Залежить від граничних значень β_t	Визначається косинусною функцією
Основна перевага	Простота реалізації та інтерпретації	Гнучкіший розподіл рівнів шуму
Застосування	Базова реалізація DDPM	Альтернативний режим дифузії

Для коректного експериментального порівняння цих режимів необхідно використовувати однаковий набір зображень, архітектуру U-Net, кількість епох навчання та спосіб оцінювання результатів.

4. Зворотний процес дифузії (Reverse Diffusion Process)

Зворотний процес дифузії – це поступове перетворення випадкового шуму або зашумленого зображення на зображення, яке має структуру, характерну для навчальних даних.

На відміну від прямого процесу, параметри якого визначаються заздалегідь, зворотний процес використовує нейронну мережу для оцінювання шуму, що необхідно усунути.

У DDPM зворотний процес описується послідовністю умовних розподілів:

$$p_\theta(x_{0:T}) = p(x_T)\prod_{t=1}^{T} p_\theta\left(x_{t-1} \mid x_t\right)$$

де $p(x_T) = \mathcal{N}(0, I)$– початковий розподіл для генерації з шуму.

Кожне зворотне перетворення моделюється нормальним розподілом:

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}\big(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t)\big)$$

де $\mu_\theta(x_t, t)$– оцінене математичне сподівання менш зашумленого зображення; $\Sigma_\theta(x_t, t)$ – коваріаційна матриця зворотного переходу; θ – параметри навченої нейронної мережі.

У спрощеній реалізації DDPM для лабораторної роботи дисперсія зворотного переходу задається заздалегідь, а нейронна мережа прогнозує шум $\varepsilon\theta(x_t,t)$.

Прогнозування шуму нейронною мережею. Під час прямого процесу зашумлене зображення формується за відомою формулою:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\varepsilon$$

Оскільки під час навчання випадковий шум відомий, його можна використати як цільове значення для нейронної мережі.

Завдання нейронної мережі полягає в побудові функції:

$$\hat{\varepsilon} = \varepsilon_\theta(x_t, t)$$

Де x_t– зашумлене зображення; t – номер кроку дифузії; $\hat{\varepsilon}$– прогнозований нейронною мережею шум.

Тобто мережа отримує не лише саме зображення, а й інформацію про рівень його зашумлення.

Це необхідно, оскільки однакові візуальні структури можуть відповідати різним етапам дифузії, а алгоритм очищення залежить від поточного кроку.

Оцінювання початкового зображення. Якщо нейронна мережа спрогнозувала шум, із формули прямого процесу можна отримати оцінку початкового зображення:

$$\hat{x}_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\varepsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}$$

Цей вираз показує, що оцінювання початкового зображення ґрунтується на відніманні передбаченої шумової складової.

Однак у стандартному DDPM для переходу від x_t до x_{t-1} використовується не просто $\hat{x}_0$, а спеціально параметризований умовний розподіл.

Формула зворотного кроку DDPM. Математичне сподівання зворотного переходу за параметризації прогнозування шуму визначається як:

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon_\theta(x_t, t)\right)$$

Після обчислення формується зображення попереднього кроку:

$$x_{t-1} = \mu_\theta(x_t, t) + \sqrt{\tilde{\beta}_t}z$$

де z ~ N(0,I), а $\tilde{\beta}_t$ – задана дисперсія зворотного переходу, яка визначається наступним чином:

$$\tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t$$

β_t – дисперсія апостеріорного розподілу прямого процесу. Для граничного випадку приймають $\bar{\alpha}_0 = 1$.

На останньому кроці t=1 випадковий шум не додається.

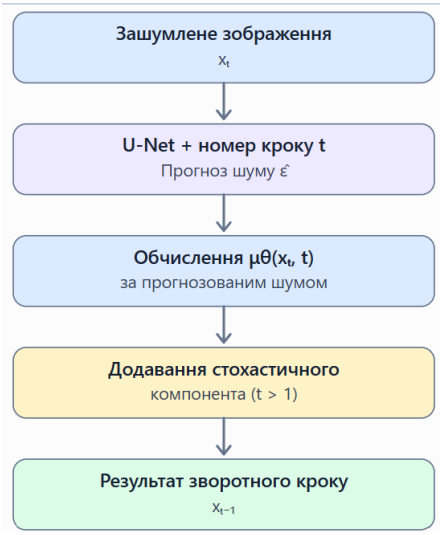


Рис. 5. Послідовність одного кроку зворотної дифузії

Таким чином, зворотний процес передбачає багаторазове використання тієї самої навченої мережі, але з різними значеннями кроку і різними вхідними зображеннями.

Генерування та відновлення зображень. У лабораторній роботі зворотний процес використовуватиметься у двох режимах.

Генерування нового зображення. Початковий стан формується випадково:

$$x_T \sim \mathcal{N}(0, I)$$

Після цього послідовно виконуються зворотні переходи до x_0. Отримане зображення є новим зразком, сформованим моделлю, а не точною копією конкретного навчального зображення.

Відновлення зашумленого зображення. Спочатку відоме зображення зашумлюється до вибраного кроку :

$$x_{t_0} = \sqrt{\bar{\alpha}_{t_0}} x_0 + \sqrt{1 - \bar{\alpha}_{t_0}} \varepsilon$$

Потім виконується зворотний процес:

$$x_{t_0} \to x_{t_0 - 1} \to \cdots \to \hat{x}_0$$

Отриманий результат порівнюється з початковим зображенням.

При цьому слід враховувати, що зворотна дифузія є стохастичною генеративною процедурою. Вона не гарантує точного відновлення кожного пікселя початкового зображення. За великого рівня шуму можливе відтворення іншого зображення, яке за структурою відповідає навчальному розподілу.

Це особливо важливо під час оцінювання MSE, PSNR і SSIM: ці метрики характеризують близькість до конкретного оригіналу, тоді як візуальна правдоподібність генерації є окремою характеристикою.

5. Архітектура U-Net для прогнозування шуму

U-Net – архітектура згорткової нейронної мережі, яка поєднує послідовне зменшення просторової розмірності представлень із подальшим відновленням їхньої роздільної здатності. Її характерною особливістю є використання пропускних з'єднань (Skip Connections) між відповідними рівнями кодувальної та декодувальної частин.

У дифузійних моделях U-Net використовується для прогнозування шуму, доданого до зображення на певному кроці дифузії.

На відміну від класичної U-Net для сегментації, яка формує карту класів пікселів, U-Net у DDPM повертає тензор шуму, що має такі самі просторові розміри й кількість каналів, як вхідне зображення.

Для одноканального зображення розміром 28×28 вихід U-Net також має розмір 28×28×1.

Основні компоненти U-Net. Архітектура складається з трьох основних частин.

Кодувальна частина (Encoder). Послідовно формує ознаки вхідного зображення та зменшує їхню просторову розмірність. Для цього використовуються згорткові шари та операції Downsampling.

На нижчих рівнях мережі виділяються локальні ознаки, зокрема контури й текстурні елементи. Глибші рівні формують представлення більших ділянок зображення.

Проміжний блок (Bottleneck). Розташований між кодувальною та декодувальною частинами й опрацьовує компактне представлення зображення. Він забезпечує додаткове нелінійне перетворення ознак.

Декодувальна частина (Decoder). Поступово збільшує просторову розмірність карт ознак за допомогою Upsampling або транспонованих згорток (ConvTranspose2d).

Під час декодування використовуються ознаки, отримані на відповідних рівнях Encoder.

Пропускні з'єднання (Skip Connections). Однією з основних особливостей U-Net є передавання карт ознак безпосередньо з кодувальної частини до відповідних рівнів декодувальної.

Зазвичай це здійснюється шляхом конкатенації:

$$F_{dec} = Concat(F_{up}, F_{enc})$$

Де F_{up}– карта ознак після збільшення розмірності; F_{enc}– карта ознак відповідного рівня Encoder; F_{dec}– об'єднання тензорів уздовж осі каналів.

Такі з'єднання дозволяють зберігати інформацію про просторове розташування деталей, яка могла бути частково втрачена під час зменшення розмірності.

Для дифузійної моделі це важливо, оскільки нейронна мережа повинна оцінювати шум для кожного пікселя, використовуючи як локальні, так і глобальні характеристики зображення.

Спрощена архітектура U-Net для лабораторної роботи. Для зображень Fashion-MNIST доцільно використати компактну архітектуру з двома рівнями зменшення просторової розмірності.

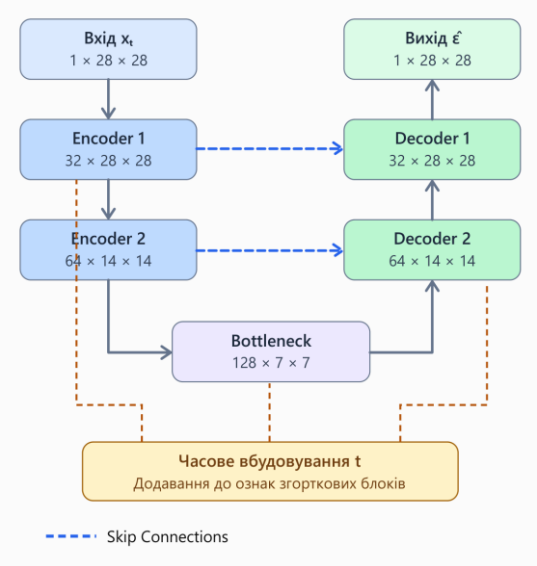


Рис. 6. Спрощена структура U-Net з двома рівнями Encoder і Decoder, пропускними з'єднаннями та врахуванням номера кроку дифузії. Для однозначності реалізації наведені розміри карт ознак; передавання часу показане схематично.

Приклад структури моделі:

Блок	Операція	Розмір виходу, C×H×W
Вхід	Зашумлене зображення	1×28×28
Encoder 1	Conv2d, SiLU	32×28×28
Downsampling 1	Conv2d, stride=2	32×14×14
Encoder 2	Conv2d, SiLU	64×14×14
Downsampling 2	Conv2d, stride=2	64×7×7
Bottleneck	Conv2d, SiLU	128×7×7
Upsampling 1	ConvTranspose2d	64×14×14
Skip Connection	Concat із Encoder 2	128×14×14
Decoder 2	Conv2d, SiLU	64×14×14
Upsampling 2	ConvTranspose2d	32×28×28
Skip Connection	Concat із Encoder 1	64×28×28
Decoder 1	Conv2d, SiLU	32×28×28
Вихід	Conv2d, kernel_size=1	1×28×28

Наведена архітектура є навчальним спрощенням U-Net. Для обробки складніших зображень можуть використовуватися додаткові згорткові блоки, залишкові з'єднання (Residual Connections) і механізми уваги (Attention).

Часове вбудовування (Time Embedding). На відміну від звичайної U-Net, дифузійна мережа повинна враховувати номер кроку t, для якого необхідно спрогнозувати шум.

Тому її вхід складається з двох компонентів (x_t, t). Номер кроку не обов'язково передається до мережі безпосередньо як одне число. Його можна перетворити на багатовимірний вектор ознак – часове вбудовування (Time Embedding).

Одним із поширених підходів є синусоїдальне кодування:

$$e_{2i}(t) = \sin\left(\frac{t}{10000^{2i/d}}\right), \quad e_{2i+1}(t) = \cos\left(\frac{t}{10000^{2i/d}}\right)$$

де d – розмірність вектора часового вбудовування, а i = 0, …, d/2 − 1.

Отриманий вектор передається до невеликої повнозв'язної мережі та перетворюється на представлення потрібної розмірності.

Далі інформація про крок дифузії додається до проміжних карт ознак згорткових блоків:

$$h' = h + Broadcast\big(W_e e(t)\big),$$

де h – карта ознак, $e(t)$ – часове вбудовування, W – навчуване лінійне перетворення, а $Broadcast$ – поширення вектора на просторові координати.

Завдяки цьому одна U-Net може прогнозувати шум на різних етапах дифузії.

У практичній реалізації буде використано синусоїдальне часове вбудовування з подальшим перетворенням через лінійні шари.

6. Навчання дифузійної моделі

Навчання DDPM спрямоване на визначення параметрів нейронної мережі θ, за яких прогнозований шум максимально наближається до шуму, що був використаний для зашумлення зображення.

На відміну від класичного навчання мережі для відновлення зображень, у DDPM цільовим значенням є не саме початкове зображення x_0, а випадковий шум ε.

Формування навчальних прикладів. На кожній ітерації навчання виконуються такі операції:

1. Із навчального набору вибирається пакет початкових зображень x_0.
2. Для кожного зображення випадково вибирається крок дифузії $t \in \{1, \ldots, T\}$.
3. Генерується випадковий шум $\varepsilon \sim N(0,I)$.
4. За формулою прямого процесу формується зашумлене зображення x_t.
5. Нейронна мережа отримує x_t і t та прогнозує $\hat{\varepsilon}$.
6. Обчислюється функція втрат і оновлюються параметри U-Net.

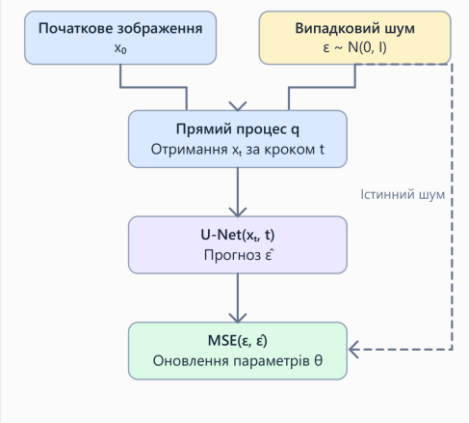


Рис. 7. Прогнозування шуму та оновлення параметрів U-Net.

Функція втрат. У спрощеній реалізації DDPM використовується середньоквадратична похибка (Mean Squared Error, MSE). Для одного зображення $D = HWC$ з елементами:

$$L_{\text{MSE}} = \frac{1}{D}\sum_{j=1}^{D}\left(\varepsilon_j - \hat{\varepsilon}_j\right)^2$$

Для навчальної вибірки функція втрат визначається як:

$$L_{\text{simple}}(\theta) = \mathbb{E}_{x_0,t,\varepsilon}[\| \varepsilon - \varepsilon_\theta(x_t, t) \|_2^2]$$

де

У програмній реалізації використовується усереднена MSE за всіма пікселями та об’єктами пакета. Ця нормалізація відрізняється від запису через квадрат евклідової норми лише масштабним множником за фіксованого розміру зображень.

Мінімізація функції втрат змушує модель прогнозувати шум на різних кроках дифузії.

6.3. Оптимізація параметрів

Параметри U-Net оновлюються методом зворотного поширення похибки (Backpropagation).

Для мінімізації функції втрат використовується оптимізатор Adam:

Після кожної ітерації обчислюється прогноз шуму, визначається MSE, виконуються обчислення градієнтів та оновлення ваг.

Для оцінювання навчання необхідно зберігати значення функції втрат і будувати графік її зміни за епохами.

Слід враховувати, що зменшення MSE прогнозування шуму не завжди означає пропорційне покращення візуальної якості згенерованих зображень. Тому результати навчання необхідно доповнювати оцінюванням зразків, отриманих у зворотному процесі.

6.4. Рекомендовані параметри навчання

Для базового прикладу лабораторної роботи пропонується така конфігурація.

Параметр	**Значення**
Набір даних	Fashion-MNIST
Розмір зображення	28×28×1
Нормалізація	[-1,1]
Архітектура	Спрощена U-Net
Кількість рівнів Downsampling	2
Кількість рівнів Upsampling	2

Параметр	Значення
Кількість каналів	32, 64, 128
Часове вбудовування	Синусоїдальне
Функція активації	SiLU
Функція втрат	MSE
Оптимізатор	Adam
Learning rate	0.001
Batch size	64
Кількість кроків дифузії	200
Розклад шуму	Linear
Кількість епох	20–30 для початкових експериментів

алгоритм генерування та відновлення зображень із використанням навченої DDPM; вплив кількості кроків дифузії та режиму додавання шуму; методи оцінювання якості – MSE, PSNR, SSIM;

порівняння режимів генерації та відновлення; підсумкову схему дослідження дифузійної моделі.

Після цього перейдемо до практичного прикладу з повним кодом на PyTorch, візуалізаціями проміжних результатів та аналізом експериментів.

Теоретичний матеріал (продовження)

7. Генерування зображень за допомогою дифузійної моделі

Генерування зображень (Image Generation) у дифузійних моделях здійснюється шляхом послідовного перетворення випадкового шуму на зображення, структура якого відповідає закономірностям навчального набору даних.

Після завершення навчання нейронна мережа U-Net використовується для прогнозування шуму на кожному кроці зворотної дифузії. На відміну від прямого процесу, у якому інформація про початкове зображення поступово руйнується, зворотний процес спрямований на формування структурованого зображення з випадкового шумового представлення.

7.1. Початковий шумовий стан

Процес генерації починається з випадкового тензора:

де – початкове шумове зображення; – загальна кількість кроків дифузії; – стандартний багатовимірний нормальний розподіл.

Розмірність випадкового тензора повинна відповідати розмірності зображень, на яких навчалася модель.

Наприклад, для Fashion-MNIST:

Кожен елемент тензора формується незалежно з нормального розподілу. На цьому етапі шумове представлення не містить визначеної візуальної структури.

7.2. Послідовне генерування зображення

Після формування початкового шуму послідовно виконуються кроки зворотної дифузії:

$$x_T \to x_{T-1} \to \cdots \to x_1 \to x_0$$

На кожному кроці нейронна мережа прогнозує шум:

Після цього обчислюється математичне сподівання зворотного переходу:

$$\hat{\varepsilon}_t = \varepsilon_\theta(x_t, t)$$

Нове представлення формується за правилом:

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\hat{\varepsilon}_t\right)$$

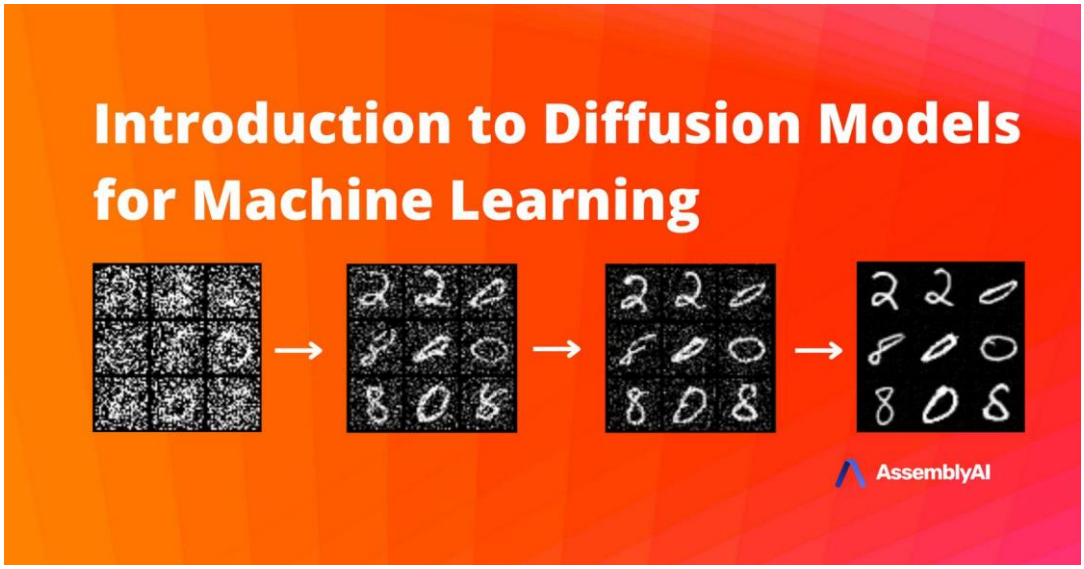


Рис. 8. Ілюстрація послідовного формування зображення під час зворотної дифузії.

об’єкта.

Однак така закономірність характерна для достатньо навченої моделі. Якщо навчання було недостатнім, кінцевий результат може залишатися нечітким або містити значні спотворення.

7.3. Стохастичність генерації

Важливою особливістю DDPM є стохастичний характер генерації.

Навіть якщо двічі запустити зворотний процес з однаковими параметрами моделі, результати можуть відрізнятися через випадковий початковий шум і випадкові компоненти зворотних переходів.

Це дозволяє формувати різноманітні зображення, що відповідають розподілу навчальної вибірки.

Для відтворюваності експериментів доцільно фіксувати генератор випадкових чисел ( ).

7.4. Візуалізація проміжних результатів

Для аналізу процесу генерації необхідно зберігати проміжні зображення після визначених кроків.

Наприклад, для можна зберегти результати на кроках:

Таке представлення дозволяє дослідити, як змінюється структура зображення в процесі видалення шуму та на яких етапах проявляються характерні ознаки об’єкта.

У лабораторній роботі необхідно реалізувати алгоритм генерації з випадкового шуму, сформувати декілька зображень, візуалізувати проміжні результати зворотного процесу та проаналізувати їхню якість.

8. Відновлення зашумлених зображень

Відновлення зображень (Image Denoising) полягає в оцінюванні початкового зображення за його зашумленим представленням.

У лабораторній роботі для цього використовується та сама навчена дифузійна модель, яка застосовувалася для генерації нових зображень.

Однак початкове шумове представлення формується не випадково, а шляхом зашумлення конкретного зображення.

8.1. Формування зашумленого зображення

Нехай задано початкове зображення x_0.

Для отримання його зашумленої версії вибирається крок дифузії , після чого обчислюється:

$$x_{t_0} = \sqrt{\bar{\alpha}_{t_0}} x_0 + \sqrt{1 - \bar{\alpha}_{t_0}}\varepsilon$$

де $\varepsilon \sim \mathcal{N}(0, I)$.

Чим менше значення $\bar{\alpha}_{t_0}$, тим меншим є внесок початкового зображення у сформований тензор.

Для дослідження впливу рівня зашумлення можна використати декілька значень:

за умови, що T = 200.

Важливо враховувати, що фактичний рівень шуму залежить не лише від номера кроку, а й від вибраного розкладу шуму.

8.2. Зворотне відновлення

Отримане зашумлене зображення використовується як початкове представлення для зворотного процесу:

На кожному кроці нейронна мережа прогнозує шум і формує менш зашумлене представлення.

Кінцевий результат є оцінкою початкового зображення.

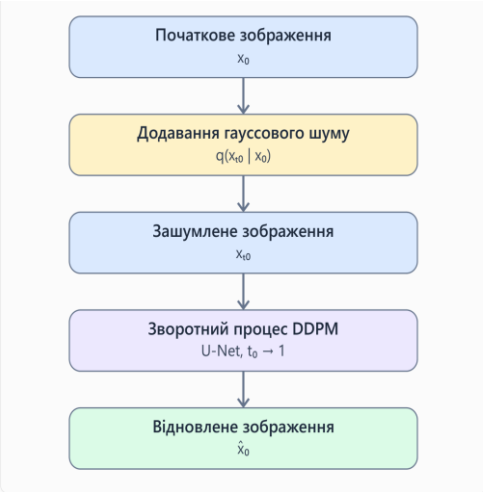


Рис. 9. Послідовність зашумлення початкового зображення та його відновлення за допомогою навченої дифузійної моделі.

8.3. Особливості відновлення

На відміну від детермінованих алгоритмів фільтрації, дифузійна модель не обов’язково відновлює всі пікселі зображення у точній відповідності до оригіналу.

Модель використовує закономірності, засвоєні під час навчання, тому відновлений результат може відрізнятися від початкового зображення.

За невеликого рівня шуму модель зазвичай має більше інформації про структуру об’єкта. За значного зашумлення частина початкової інформації втрачається, що ускладнює точне відновлення.

Зокрема, можуть змінюватися дрібні деталі, форма контурів або інші особливості зображення.

Для об’єктивного оцінювання необхідно порівнювати відновлене зображення з початковим за кількісними метриками, а також виконувати візуальний аналіз.

Варто розрізняти відновлення після відомого дифузійного зашумлення та очищення довільного реального шумового зображення. У другому випадку необхідно враховувати статистичні характеристики шуму, які можуть відрізнятися від навчального гауссового процесу.

9. Вплив кількості кроків дифузії та розкладу шуму

Якість генерації та відновлення зображень залежить від параметрів прямого і зворотного процесів. Основними з них є загальна кількість кроків дифузії , розклад шуму , початковий рівень зашумлення та кількість кроків зворотного процесу.

9.1. Кількість кроків дифузії

Параметр визначає кількість дискретних переходів прямого процесу.

Для порівняльного дослідження можна використати:

Менша кількість кроків може зменшувати обчислювальні витрати, але за невдало підібраного розкладу шуму окремі переходи стають складнішими для моделювання.

Збільшення кількості кроків дозволяє розподілити процес зашумлення та відновлення на дрібніші переходи, проте потребує більшої кількості обчислень під час стандартного DDPM sampling.

Для кожного значення необхідно сформувати відповідний розклад шуму та забезпечити узгодженість процесів навчання й генерації.

Важливо, що модель, навчена для конкретного розкладу шуму та діапазону часових кроків, не повинна довільно використовувати інший розклад без відповідного узгодження.

9.2. Кількість кроків зворотного процесу

У класичному DDPM зворотне семплювання використовує всі кроки:

Однак існують прискорені алгоритми, зокрема Denoising Diffusion Implicit Models (DDIM), які дозволяють використовувати меншу кількість кроків генерації.

Тому важливо розрізняти:

зміну загальної кількості кроків під час навчання окремих моделей; скорочення кількості кроків семплювання вже навченої моделі за допомогою відповідного алгоритму.

У базовій лабораторній роботі пропонується реалізувати стандартний DDPM sampling, а прискорене семплювання можна розглядати як додаткове індивідуальне дослідження.

9.3. Вплив розкладу шуму

$$\mathrm{SNR}(t) = \frac{\bar{\alpha}_t}{1 - \bar{\alpha}_t}$$

Вибраний розклад шуму визначає, скільки інформації початкового зображення зберігається на кожному кроці.

Для порівняння лінійного та косинусного розкладів необхідно оцінювати не лише значення β_t, а й накопичений коефіцієнт:

Додатково можна використати відношення сигналу до шуму:

де характеризує співвідношення дисперсій сигналу та шумової складової за умови одиничної дисперсії нормалізованого сигналу.

Високе значення SNR відповідає переважанню сигналу, низьке – переважанню шуму.

Під час дослідження студент повинен пояснити, як зміна , та SNR впливає на зашумлення зображень і складність їхнього відновлення.

10. Оцінювання якості генерації та відновлення

Оцінювання результатів дифузійної моделі повинно враховувати два аспекти: точність відновлення конкретного зображення та візуальну якість сформованих зразків.

Для відновлення зображень із відомим оригіналом використовуються метрики MSE, PSNR та SSIM.

10.1. Середньоквадратична похибка (MSE)

Mean Squared Error характеризує середній квадрат різниці між пікселями початкового та відновленого зображень:

де – початкове зображення, – відновлене зображення.

$$\mathrm{MSE} = \frac{1}{HWC}\sum_{i=1}^{H}\sum_{j=1}^{W}\sum_{c=1}^{C}\left(x_{ijc} - \hat{x}_{ijc}\right)^2$$

Менше значення MSE відповідає меншій середній квадратичній різниці між зображеннями.

Недоліком цієї метрики є те, що вона не повністю враховує особливості людського сприйняття зображень.

10.2. Пікове відношення сигналу до шуму (PSNR)

Peak Signal-to-Noise Ratio використовується для оцінювання якості відновлених зображень відносно оригіналу.

Метрика визначається як:

$$\mathrm{PSNR} = 10\log_{10}\left(\frac{MAX_I^2}{\mathrm{MSE}}\right)$$

де – максимальна амплітуда сигналу відповідно до використовуваного діапазону значень пікселів.

Для зображень, представлених у діапазоні [0,1], використовується MAX_I = 1.

Якщо дані залишаються нормалізованими у діапазоні , необхідно враховувати динамічний діапазон, що дорівнює 2.

У лабораторній роботі для обчислення PSNR доцільно перетворювати оригінальні та відновлені зображення до діапазону [0,1].

Більше значення PSNR відповідає меншій похибці відносно оригіналу. Якщо MSE дорівнює нулю, PSNR прямує до нескінченності.

10.3. Індекс структурної подібності (SSIM)

$$\mathrm{SSIM}(x, \hat{x}) = \frac{(2\mu_x\mu_{\hat{x}} + C_1)(2\sigma_{x\hat{x}} + C_2)}{\left(\mu_x^2 + \mu_{\hat{x}}^2 + C_1\right)\left(\sigma_x^2 + \sigma_{\hat{x}}^2 + C_2\right)}$$

Structural Similarity Index Measure оцінює подібність двох зображень з урахуванням яскравості, контрасту та локальної структури.

Загальний вигляд метрики:

де:

– середні значення яскравості; – дисперсії;

– коваріація;

– сталі для числової стійкості.

У стандартному випадку значення SSIM, близьке до 1, відповідає високій структурній подібності зображень.

На відміну від MSE, SSIM враховує локальні структурні властивості, тому краще відображає деякі аспекти візуальної якості.

10.4. Порівняння метрик

Метрика	Що оцінює	Інтерпретація
MSE	Середню квадратичну похибку пікселів	Менше – краще
PSNR	Відношення максимальної потужності сигналу до похибки	Більше – краще
SSIM	Подібність яскравості, контрасту та структури	Ближче до 1 – краще

зображеннях і в однаковому діапазоні значень пікселів.

10.5. Оцінювання генерації нових зображень

На відміну від задачі відновлення, під час безумовної генерації нового зображення відсутній конкретний оригінал, з яким можна безпосередньо порівняти результат.

Тому MSE, PSNR та SSIM не є достатніми метриками для оцінювання безумовної генерації.

У базовій лабораторній роботі якість згенерованих зображень пропонується оцінювати візуально за такими критеріями:

наявність характерної форми об’єктів Fashion-MNIST; чіткість основних контурів; наявність помітних спотворень і шумових артефактів; різноманітність згенерованих зображень.

Для розширеного оцінювання генеративних моделей можуть використовуватися Fréchet Inception Distance (FID) або Kernel Inception Distance (KID). Однак для невеликого навчального експерименту з Fashion-MNIST їх застосування потребує додаткового налаштування мережі виділення ознак та достатньої кількості зразків.

11. Загальна схема дослідження дифузійної моделі

У лабораторній роботі необхідно реалізувати прямий та зворотний процеси дифузії, виконати навчання U-Net для прогнозування шуму, після чого застосувати модель у двох режимах: генерування нових зображень і відновлення зашумлених.

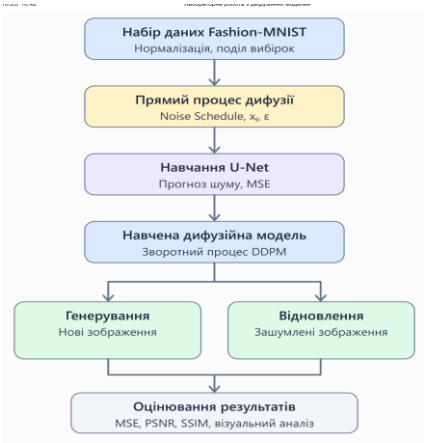


Рис. 10. Основні етапи реалізації, навчання та експериментального дослідження дифузійної моделі.

Отримані результати аналізуються за якістю зображень, значеннями кількісних метрик та

обчислювальними витратами.

Усі порівняльні дослідження повинні виконуватися з контролем інших параметрів, щоб можна було визначити вплив саме тієї характеристики, яку змінюють у відповідному експерименті.

Приклад. Побудова дифузійної моделі DDPM для генерування та відновлення зображень

Завдання. Побудувати спрощену дифузійну модель DDPM (Denoising Diffusion Probabilistic Model) для набору зображень Fashion-MNIST. Реалізувати прямий процес дифузії шляхом поступового додавання гауссового шуму, побудувати згорткову нейронну мережу U-Net для прогнозування шуму та виконати її навчання. Використати навчену модель для генерування нових і відновлення зашумлених зображень. Оцінити якість відновлення за метриками MSE, PSNR та SSIM.

Для реалізації використовується бібліотека TensorFlow/Keras. Приклад розрахований на виконання у Google Colab або іншому середовищі Python із підтримкою TensorFlow.

1. Підключення бібліотек і налаштування середовища

Для побудови моделі використовуються TensorFlow та Keras, для числових обчислень – NumPy, для візуалізації – Matplotlib.

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow import keras
from tensorflow.keras import layers

tf.random.set_seed(42)
np.random.seed(42)

print("TensorFlow:", tf.__version__)
print("GPU:", tf.config.list_physical_devices("GPU"))
```

Для пришвидшення навчання рекомендується використовувати GPU.

У Google Colab його можна підключити через меню Runtime → Change runtime type → GPU.

Код не вимагає обов'язкової наявності GPU, однак навчання згорткової мережі на CPU може потребувати значно більше часу.

2. Завантаження та підготовка Fashion-MNIST

Fashion-MNIST – набір зображень предметів одягу, взуття й аксесуарів, що містить 60 000 навчальних і 10 000 тестових зображень.

Кожне зображення має розмір пікселів і один канал яскравості.

Для роботи дифузійної моделі значення пікселів нормалізуються до діапазону :

$$x_{\text{norm}} = \frac{x}{127.5} - 1$$

Така нормалізація дозволяє використовувати зображення із симетричним відносно нуля діапазоном значень.

Завантаження та нормалізація

```
(x_train, _), (x_test, _) = (
    keras.datasets.fashion_mnist.load_data()
)

x_train = (
    x_train.astype("float32") / 127.5 - 1.0
)

x_test = (
    x_test.astype("float32") / 127.5 - 1.0
```

```
)

x_train = np.expand_dims(x_train, axis=-1)
x_test = np.expand_dims(x_test, axis=-1)

print("Train:", x_train.shape)
print("Test:", x_test.shape)
```

Очікувані розмірності:

```
Train: (60000, 28, 28, 1)
Test:  (10000, 28, 28, 1)
```

Для навчання використаємо пакети розміром 64 зображення.

```
BATCH_SIZE = 64

train_ds = (
    tf.data.Dataset.from_tensor_slices(x_train)
    .shuffle(60000, seed=42)
    .batch(BATCH_SIZE)
    .prefetch(tf.data.AUTOTUNE)
)
```

Візуалізація початкових зображень

```
fig, axes = plt.subplots(2, 5, figsize=(10, 5))

for i, ax in enumerate(axes.flat):
    image = (x_train[i, :, :, 0] + 1) / 2

    ax.imshow(image, cmap="gray", vmin=0, vmax=1)
    ax.axis("off")

plt.suptitle("Початкові зображення Fashion-MNIST")
plt.tight_layout()
plt.show()
```

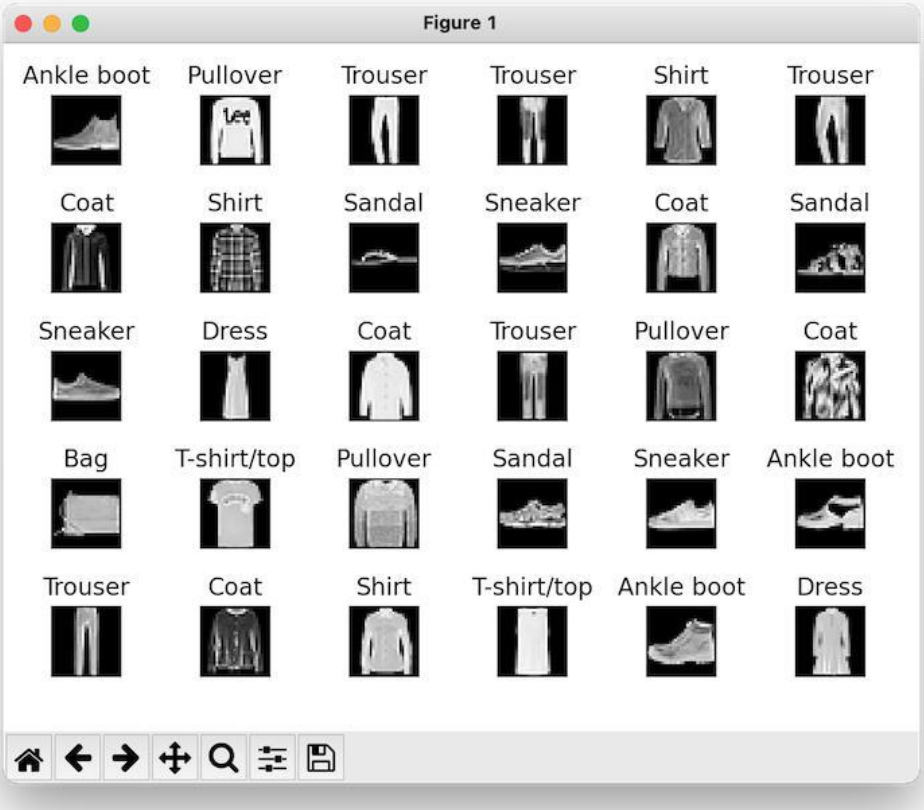


Рис. 11. Приклад початкових зображень Fashion-MNIST.

дослідження роботи дифузійної моделі з невеликою U-Net.

3. Формування розкладу шуму

Для моделювання прямого процесу необхідно визначити кількість кроків дифузії T та послідовність параметрів β_t.

У базовому прикладі використаємо:

Реалізуємо два варіанти розкладу: Linear та Cosine.

Лінійний і косинусний розклади

```
T = 200

def linear_beta_schedule(
    timesteps,
    beta_start=1e-4,
    beta_end=0.02
):
    return np.linspace(
        beta_start,
        beta_end,
        timesteps,
        dtype=np.float64
    )


def cosine_beta_schedule(timesteps, s=0.008):
    steps = np.arange(
        timesteps + 1, dtype=np.float64
    )

    f = np.cos(
        ((steps / timesteps + s) / (1 + s))
        * np.pi / 2
    ) ** 2

    alpha_bar = f / f[0]

    betas = 1 - (
        alpha_bar[1:] / alpha_bar[:-1]
    )

    return np.clip(betas, 1e-8, 0.999)
```

Для базового навчання оберемо лінійний розклад.

```
SCHEDULE = "linear"

if SCHEDULE == "linear":
    betas_np = linear_beta_schedule(T)
else:
    betas_np = cosine_beta_schedule(T)

alphas_np = 1.0 - betas_np

alpha_bar_np = np.cumprod(alphas_np)

alpha_bar_prev_np = np.concatenate(
    ([1.0], alpha_bar_np[:-1])
)

betas = tf.constant(
    betas_np, dtype=tf.float32
)

alphas = tf.constant(
    alphas_np, dtype=tf.float32
)

alpha_bar = tf.constant(
    alpha_bar_np, dtype=tf.float32
)

alpha_bar_prev = tf.constant(
    alpha_bar_prev_np, dtype=tf.float32
)
```

У програмі використовується індексація t = 0, ..., T-1, яка відповідає математичним крокам

.

Візуалізація розкладу шуму

```
linear_betas = linear_beta_schedule(T)
cosine_betas = cosine_beta_schedule(T)

linear_alpha_bar = np.cumprod(1 - linear_betas)
cosine_alpha_bar = np.cumprod(1 - cosine_betas)

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(
    linear_betas, label="Linear"
)
axes[0].plot(
    cosine_betas, label="Cosine"
)

axes[0].set_title("Розклад шуму")
axes[0].set_xlabel("Крок дифузії")
axes[0].set_ylabel("β")
axes[0].legend()
axes[0].grid(True)

axes[1].plot(
    linear_alpha_bar, label="Linear"
)
axes[1].plot(
    cosine_alpha_bar, label="Cosine"
)

axes[1].set_title("Збереження сигналу")
axes[1].set_xlabel("Крок дифузії")
axes[1].set_ylabel("alpha_bar")
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.show()
```

За результатами побудови графіків необхідно порівняти характер зміни та для двох розкладів. Слід звернути увагу, що за косинусного розкладу значення можуть різко зростати наприкінці процесу.

4. Реалізація прямого процесу дифузії

Прямий процес дифузії використовується для формування зашумлених зображень на довільному кроці t.

Для цього застосовується формула:

$$\text{де } \varepsilon \sim \mathcal{N}(0, I).$$

У реалізації потрібно враховувати, що в одному пакеті кожне зображення може мати свій номер кроку дифузії.

Функція прямого зашумлення

```
def forward_diffusion(x0, t, noise=None):
    if noise is None:
        noise = tf.random.normal(tf.shape(x0))

    alpha_bar_t = tf.gather(alpha_bar, t)

    alpha_bar_t = tf.reshape(
        alpha_bar_t, (-1, 1, 1, 1)
    )

    xt = (
        tf.sqrt(alpha_bar_t) * x0
        +
        tf.sqrt(1.0 - alpha_bar_t) * noise
```

```
    )

    return xt, noise
```

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t}\varepsilon$$

Функція отримує початкові зображення x0, номери кроків t і необов'язковий тензор шуму noise.

Якщо шум не задано, він генерується за допомогою tf.random.normal().

Функція повертає два тензори: зашумлені зображення та шум, який використовувався для їх формування. Другий тензор буде цільовим значенням під час навчання U-Net.

Візуалізація прямої дифузії

```
image = tf.convert_to_tensor(
    x_test[0:1], dtype=tf.float32
)

steps = [0, 24, 49, 99, 149, 199]

# Однакова реалізація шуму для порівняння
fixed_noise = tf.random.normal(
    tf.shape(image), seed=42
)

fig, axes = plt.subplots(
    1, len(steps) + 1, figsize=(14, 3)
)

axes[0].imshow(
    (image[0, :, :, 0].numpy() + 1) / 2,
    cmap="gray", vmin=0, vmax=1
)
axes[0].set_title("Оригінал")
axes[0].axis("off")

for i, step in enumerate(steps):
    t = tf.constant([step], dtype=tf.int32)

    xt, _ = forward_diffusion(
        image, t, fixed_noise
    )

    axes[i + 1].imshow(
        xt[0, :, :, 0],
        cmap="gray",
        vmin=-1.5,
        vmax=1.5
    )

    axes[i + 1].set_title(f"t = {step + 1}")
    axes[i + 1].axis("off")

plt.tight_layout()
plt.show()
```

Виберемо одне початкове зображення та сформуємо його зашумлені версії для різних кроків.

Для візуалізації зашумлених зображень використовується спільна шкала яскравості, щоб порівняння рівнів шуму було наочнішим.

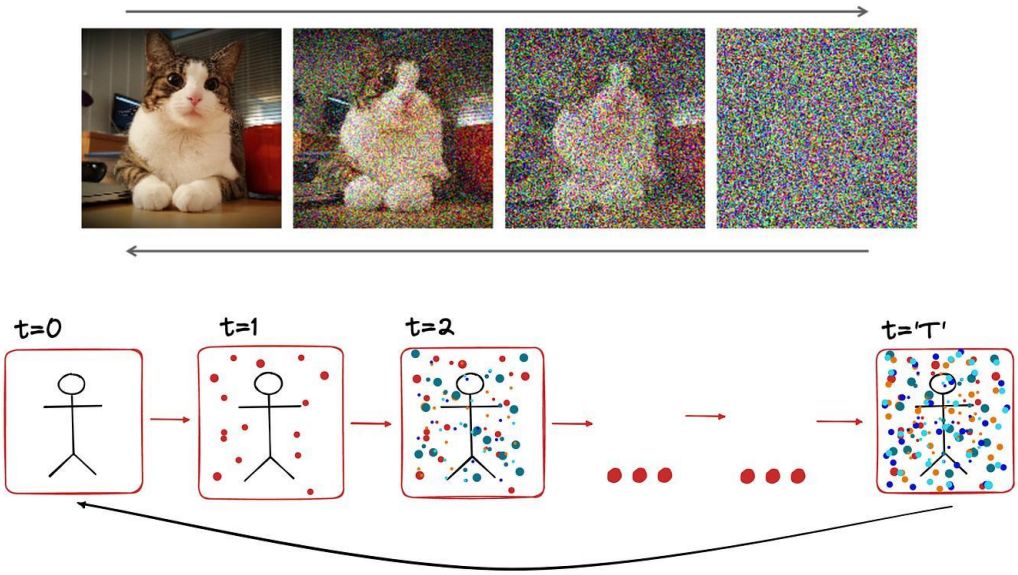


Рис. 12. Ілюстрація очікуваного характеру зміни зображення під час прямого процесу дифузії.

Аналіз результатів. На малих кроках більша частина структури початкового зображення зберігається. Зі збільшенням номера кроку зростає вплив випадкового шуму. На пізніх етапах контури об'єкта стають менш помітними.

Для вибраного лінійного розкладу з $T = 200$ наприкінці процесу може залишатися помітна частина початкового сигналу. Цю особливість необхідно враховувати під час подальшої генерації з чистого гауссового шуму.

5. Побудова спрощеної U-Net для прогнозування шуму

Основним компонентом зворотного дифузійного процесу є нейронна мережа, яка прогнозує шум, доданий до зображення на певному кроці:

$$\hat{\varepsilon} = \varepsilon_\theta(x_t, t)$$

Для цього використаємо спрощену архітектуру U-Net, що складається з кодувальної частини (Encoder), проміжного блоку (Bottleneck), декодувальної частини (Decoder) та пропускних з'єднань (Skip Connections).

Особливістю дифузійної U-Net є врахування номера кроку через часове вбудовування (Time Embedding).

Реалізація часового вбудовування

Для кодування номера кроку дифузії використовується синусоїдальне представлення.

```
class SinusoidalTimeEmbedding(layers.Layer):

    def __init__(self, dim=64):
        super().__init__()
        self.dim = dim

    def call(self, t):
        t = tf.cast(t, tf.float32)
        t = tf.reshape(t, [-1, 1])

        half_dim = self.dim // 2

        frequencies = tf.exp(
            -np.log(10000.0)
            * tf.cast(tf.range(half_dim), tf.float32)
            / half_dim
        )

        angles = t * frequencies[None, :]
```

```
        return tf.concat(
            [tf.sin(angles), tf.cos(angles)],
            axis=-1
        )
```

Цей шар перетворює скалярний номер кроку на вектор розмірності 64. Отримані ознаки дозволяють мережі враховувати рівень зашумлення під час прогнозування шуму.

Згортковий блок із часовим вбудовуванням

У згортковому блоці часове представлення перетворюється за допомогою Dense-шару та додається до карт ознак.

```
def conv_block(x, t_emb, filters):

    h = layers.Conv2D(
        filters, 3, padding="same"
    )(x)

    h = layers.Activation(tf.nn.silu)(h)

    time_projection = layers.Dense(
        filters
    )(t_emb)

    time_projection = layers.Reshape(
        (1, 1, filters)
    )(time_projection)

    h = layers.Add()([h, time_projection])

    h = layers.Conv2D(
        filters, 3, padding="same"
    )(h)

    h = layers.Activation(tf.nn.silu)(h)

    return h
```

У блоці використовуються дві згорткові операції з ядром та функцією активації SiLU.

Функція активації SiLU визначається як:

де – сигмоїдна функція.

Додавання часового представлення виконується після першої згортки. Оскільки часове представлення має форму , воно поширюється на всі просторові позиції карти ознак під час операції додавання.

Реалізація архітектури U-Net

Для зображень Fashion-MNIST використаємо два рівні зменшення та збільшення просторової розмірності.

```
def build_unet():

    image_input = keras.Input(
        shape=(28, 28, 1),
        name="noisy_image"
    )

    time_input = keras.Input(
        shape=(),
        dtype=tf.int32,
        name="timestep"
    )

    # Часове вбудовування
    t_emb = SinusoidalTimeEmbedding(64)(
        time_input
    )

    t_emb = layers.Dense(
        128, activation=tf.nn.silu
    )(t_emb)
```

```
    t_emb = layers.Dense(128)(t_emb)

    # Encoder
    e1 = conv_block(image_input, t_emb, 32)

    d1 = layers.Conv2D(
        32, 3, strides=2, padding="same"
    )(e1)

    e2 = conv_block(d1, t_emb, 64)

    d2 = layers.Conv2D(
        64, 3, strides=2, padding="same"
    )(e2)

    # Bottleneck
    b = conv_block(d2, t_emb, 128)

    # Decoder
    u2 = layers.Conv2DTranspose(
        64, 3, strides=2, padding="same"
    )(b)

    u2 = layers.Concatenate()([u2, e2])

    u2 = conv_block(u2, t_emb, 64)

    u1 = layers.Conv2DTranspose(
        32, 3, strides=2, padding="same"
    )(u2)

    u1 = layers.Concatenate()([u1, e1])

    u1 = conv_block(u1, t_emb, 32)

    # Прогноз шуму
    output = layers.Conv2D(
        1, 1, padding="same"
    )(u1)

    return keras.Model(
        inputs=[image_input, time_input],
        outputs=output,
        name="Diffusion_UNet"
    )
```

Створення моделі та аналіз архітектури

```
model = build_unet()

model.summary()
```

У результаті виконання відображається послідовність шарів мережі, розмірність вихідних тензорів та кількість параметрів.

Основні розмірності даних:

Етап	**Розмірність тензора**
Вхідне зображення	28×28×1
Encoder 1	28×28×32
Encoder 2	14×14×64
Bottleneck	7×7×128
Decoder 2	14×14×64
Decoder 1	28×28×32
Прогнозований шум	28×28×1

Decoder 128×28×32

Прогнозований шум28×28×1

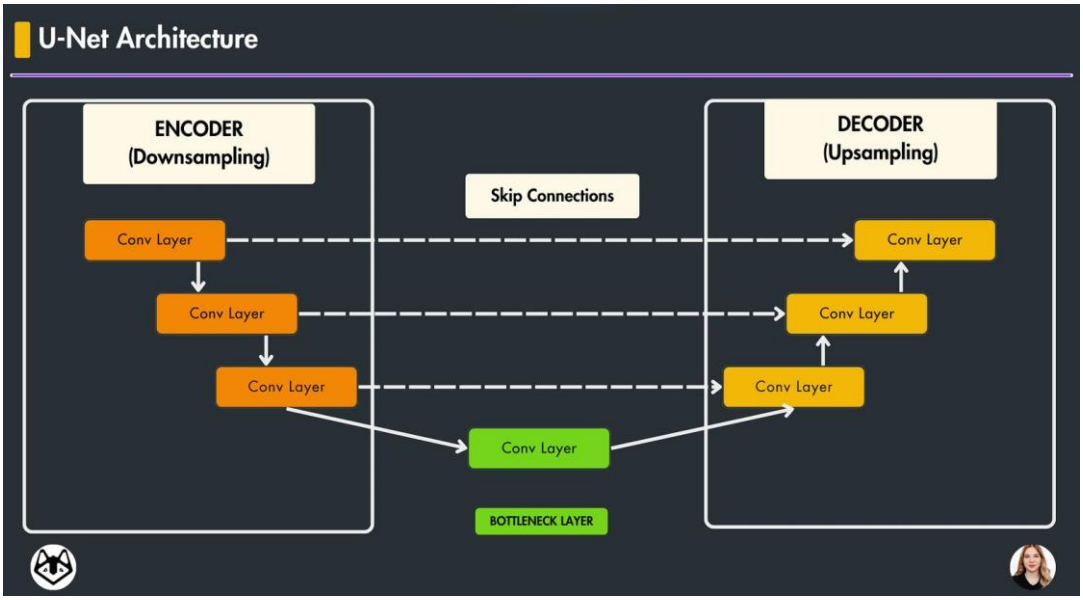


Рис. 13. Загальний принцип побудови архітектури U-Net з кодувальною, декодувальною частинами та пропускними з'єднаннями.

Аналіз архітектури. Просторова розмірність карт ознак спочатку зменшується з 28×28 до 7×7, а потім відновлюється. Пропускні з'єднання дозволяють передавати інформацію з кодувальної частини до декодувальної.

Вихідний шар Conv2D(1,1) не використовує функції активації, оскільки модель повинна прогнозувати неперервні значення гауссового шуму, які не обмежені інтервалом [0,1] або [-1,1].

6. Навчання дифузійної моделі

Навчання U-Net полягає в мінімізації різниці між шумом, доданим до початкового зображення, та шумом, спрогнозованим нейронною мережею.

Як функцію втрат використаємо MSE:

$$L = \frac{1}{N_{\text{elements}}} \sum (\varepsilon - \hat{\varepsilon})^2$$

Під час кожної навчальної ітерації випадково вибираються номери кроків дифузії, генерується гауссовий шум і формуються зашумлені зображення.

Налаштування оптимізатора

```
optimizer = keras.optimizers.Adam(
    learning_rate=1e-3
)

loss_fn = keras.losses.MeanSquaredError()
```

Реалізація одного кроку навчання

```
@tf.function
def train_step(x0):

    batch_size = tf.shape(x0)[0]

    # Випадкові номери кроків дифузії
    t = tf.random.uniform(
        shape=(batch_size,),
        minval=0,
        maxval=T,
        dtype=tf.int32
    )

    # Генерація шуму
    noise = tf.random.normal(tf.shape(x0))
```

```
    # Прямий процес дифузії
    xt, _ = forward_diffusion(x0, t, noise)

    with tf.GradientTape() as tape:

        # Прогноз шуму моделлю
        predicted_noise = model(
            [xt, t], training=True
        )

        # Обчислення функції втрат
        loss = loss_fn(
            noise, predicted_noise
        )

    # Градієнти функції втрат
    gradients = tape.gradient(
        loss, model.trainable_variables
    )

    # Оновлення параметрів
    optimizer.apply_gradients(
        zip(gradients, model.trainable_variables)
    )

    return loss
```

Декоратор забезпечує трасування обчислень у граф TensorFlow, що зазвичай підвищує швидкість виконання порівняно з покроковим виконанням лише в режимі Eager Execution.

відстежує операції, які беруть участь в обчисленні функції втрат, і дозволяє визначити градієнти за навчуваними параметрами мережі.

Основний цикл навчання

```
EPOCHS = 20

loss_history = []

for epoch in range(EPOCHS):

    epoch_losses = []

    for batch in train_ds:

        loss = train_step(batch)

        epoch_losses.append(
            float(loss.numpy())
        )

    mean_loss = np.mean(epoch_losses)

    loss_history.append(mean_loss)

    print(
        f"Epoch {epoch + 1}/{EPOCHS}, "
        f"Loss: {mean_loss:.6f}"
    )
```

У цьому прикладі використовується власний цикл навчання з , оскільки зашумлені зображення та цільові значення шуму формуються динамічно для кожного пакета.

Це дозволяє змінювати кроки дифузії під час навчання й контролювати процес формування навчальних даних.

Кількість епох може бути збільшена, якщо отримані результати залишаються незадовільними.

Візуалізація процесу навчання

```
plt.figure(figsize=(8, 5))

plt.plot(
    range(1, EPOCHS + 1),
```

```
    loss_history,
    marker="o"
)

plt.xlabel("Епоха")
plt.ylabel("MSE")
plt.title("Навчання дифузійної моделі")
plt.grid(True)

plt.show()
```

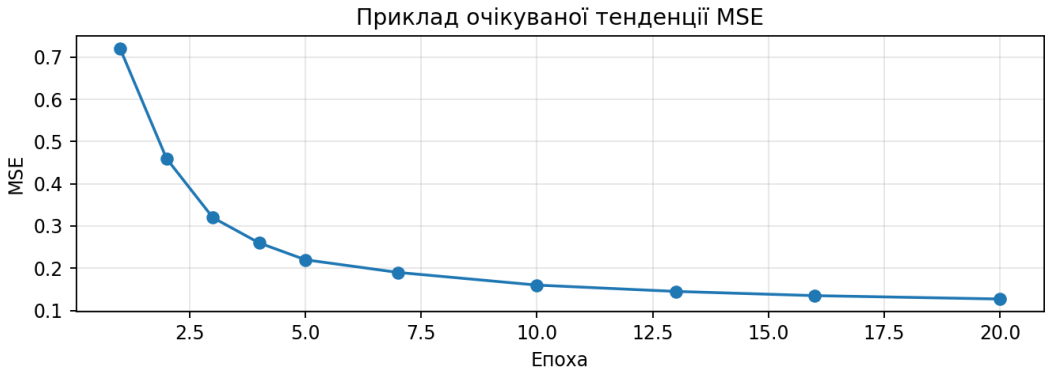


Рис. 14. Ілюстрація зменшення MSE під час навчання U-Net

доданий до зображення. Форма кривої залежить від архітектури U-Net, кількості епох, параметрів оптимізатора та розкладу шуму.

Навіть за зменшення функції втрат необхідно додатково перевіряти якість генерації зображень, оскільки мінімальне значення MSE не гарантує найкращих візуальних результатів.

Збереження навченої моделі

```
model.save_weights(
    "ddpm_unet_fashion.weights.h5"
)
```

Після збереження ваг модель можна повторно використати без нового навчання, створивши ту саму архітектуру та завантаживши ваги:

```
model = build_unet()

model.load_weights(
    "ddpm_unet_fashion.weights.h5"
)
```

Для коректного повторного використання необхідно також зберігати значення T, параметри розкладу шуму та спосіб нормалізації зображень.

7. Реалізація зворотного процесу дифузії

Після навчання U-Net можна використовувати для прогнозування шуму та послідовного отримання менш зашумлених зображень.

Для одного кроку зворотної дифузії використовується:

Для реалізації необхідно обчислити дисперсії зворотних переходів:

```
posterior_variance = (
    betas
    * (1.0 - alpha_bar_prev)
    / (1.0 - alpha_bar)
)
```

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon_\theta(x_t, t)\right) + \sqrt{\tilde{\beta}_t}z$$

Функція одного зворотного кроку

```
def reverse_step(xt, step):

    batch_size = tf.shape(xt)[0]

    t = tf.fill([batch_size], step)

    # Прогноз шуму
    predicted_noise = model(
        [xt, t], training=False
    )

    beta_t = betas[step]
    alpha_t = alphas[step]
    alpha_bar_t = alpha_bar[step]

    # Математичне сподівання
    mean = (
        xt
        -
        beta_t
        / tf.sqrt(1.0 - alpha_bar_t)
        * predicted_noise
    ) / tf.sqrt(alpha_t)

    # Останній крок без випадкового шуму
    if step == 0:
        return mean

    noise = tf.random.normal(tf.shape(xt))

    variance = posterior_variance[step]

    return (
        mean
        + tf.sqrt(variance) * noise
    )
```

Значення step = 0 відповідає останньому математичному переходу від x_1 до x_0.

На цьому кроці додатковий випадковий шум не формується.

8. Генерування нових зображень

Для генерації нових зразків спочатку формується випадковий гауссовий шум відповідної розмірності.

Після цього послідовно застосовується навчена U-Net на всіх кроках зворотного процесу.

Функція генерації

```
def generate_images(num_images=10):

    # Початковий гауссовий шум
    x = tf.random.normal(
        (num_images, 28, 28, 1)
    )

    history = {}

    save_steps = {
        T - 1, int(0.75 * T),
        int(0.5 * T), int(0.25 * T),
        0
    }

    for step in reversed(range(T)):

        x = reverse_step(x, step)

        if step in save_steps:
            history[step] = x.numpy().copy()

    return x.numpy(), history
```

Генерування та візуалізація

```
generated_images, generation_history = (
    generate_images(num_images=10)
)

fig, axes = plt.subplots(2, 5, figsize=(10, 5))

for i, ax in enumerate(axes.flat):

    image = np.clip(
        (generated_images[i, :, :, 0] + 1) / 2,
        0, 1
    )

    ax.imshow(
        image, cmap="gray",
        vmin=0, vmax=1
    )
    ax.axis("off")

plt.suptitle("Зображення, згенеровані DDPM")
plt.tight_layout()
plt.show()
```

Візуалізація проміжних етапів генерації

```
steps_to_show = sorted(
    generation_history.keys(),
    reverse=True
)

fig, axes = plt.subplots(
    1, len(steps_to_show), figsize=(13, 3)
)

for i, step in enumerate(steps_to_show):

    image = generation_history[step][0, :, :, 0]

    axes[i].imshow(
        np.clip((image + 1) / 2, 0, 1),
        cmap="gray", vmin=0, vmax=1
    )

    axes[i].set_title(
        f"Залишилось {step} кроків"
    )

    axes[i].axis("off")

plt.tight_layout()
plt.show()
```



Рис. 15. Ілюстрація поступового формування зображення з випадкового шуму під час зворотної дифузії.

контурів, різноманітність сформованих об’єктів та наявність шумових артефактів.

Важливо враховувати, що за базового лінійного розкладу з $T = 200$ кінцеве пряме зашумлення не обов’язково наближається достатньо близько до стандартного гауссового

розподілу. Тому якість безумовної генерації з $x_T \sim \mathcal{N}(0, I)$ може бути обмеженою. Для дослідження цього ефекту доцільно порівняти результат із моделлю, навченою з косинусним розкладом.

9. Відновлення зашумлених зображень

На відміну від генерації, у задачі відновлення початковий стан зворотного процесу формується з конкретного тестового зображення.

Спочатку до нього додається шум відповідно до вибраного кроку t_0, а потім модель виконує послідовні зворотні переходи.

Функція відновлення

```
def restore_images(x0, noise_step=99):

    x0 = tf.convert_to_tensor(
        x0, dtype=tf.float32
    )

    batch_size = tf.shape(x0)[0]

    t = tf.fill(
        [batch_size], noise_step
    )

    # Зашумлення початкових зображень
    noisy_images, _ = forward_diffusion(
        x0, t
    )

    x = noisy_images

    # Зворотний процес
    for step in reversed(range(noise_step + 1)):
        x = reverse_step(x, step)

    return (
        noisy_images.numpy(),
        x.numpy()
    )
```

Візуалізація результатів відновлення

```
original_images = x_test[:5]

noisy_images, restored_images = restore_images(
    original_images,
    noise_step=99
)

fig, axes = plt.subplots(3, 5, figsize=(12, 7))

titles = [
    "Оригінал",
    "Зашумлене",
    "Відновлене"
]

image_sets = [
    original_images,
    noisy_images,
    restored_images
]

for row in range(3):
    for col in range(5):

        image = np.clip(
            (image_sets[row][col, :, :, 0] + 1) / 2,
            0, 1
        )

        axes[row, col].imshow(
            image, cmap="gray",
            vmin=0, vmax=1
```

```
            )

            axes[row, col].axis("off")

            if col == 0:
                axes[row, col].set_ylabel(
                    titles[row], fontsize=11
                )

plt.tight_layout()
plt.show()
```

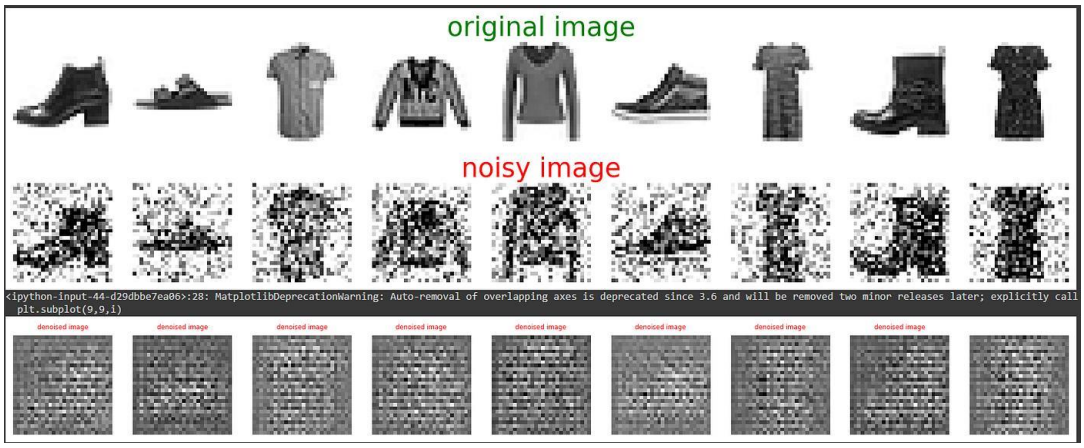


Рис. 16. Ілюстрація порівняння початкових, зашумлених і відновлених зображень.

форми об’єкта, точність контурів і залишкові шумові артефакти.

Результат відновлення може відрізнятися від оригіналу, оскільки DDPM використовує стохастичний зворотний процес і закономірності навчального розподілу.

10. Обчислення метрик якості відновлення

Для кількісного оцінювання відновлення використаємо MSE, PSNR та SSIM.

Перед обчисленням метрик усі зображення перетворюються до діапазону

```
def evaluate_restoration(original, restored):

    original = tf.convert_to_tensor(
        original, dtype=tf.float32
    )

    restored = tf.convert_to_tensor(
        restored, dtype=tf.float32
    )

    original = tf.clip_by_value(
        (original + 1) / 2, 0, 1
    )

    restored = tf.clip_by_value(
        (restored + 1) / 2, 0, 1
    )

    mse = tf.reduce_mean(
        tf.square(original - restored)
    )

    psnr = tf.reduce_mean(
        tf.image.psnr(
            original, restored, max_val=1.0
        )
    )

    ssim = tf.reduce_mean(
        tf.image.ssim(
            original, restored, max_val=1.0
        )
    )

    return {
```

```
        "MSE": float(mse.numpy()),
        "PSNR": float(psnr.numpy()),
        "SSIM": float(ssim.numpy())
    }

    .
```

Оцінювання відновлених зображень

```
metrics = evaluate_restoration(
    original_images,
    restored_images
)

for name, value in metrics.items():
    print(f"{name}: {value:.4f}")
```

Значення метрик залежать від фактично навченої моделі та рівня шуму. Вони повинні бути отримані студентом під час виконання програми.

Дослідження впливу рівня зашумлення

Для порівняння використаємо чотири рівні зашумлення. Під час кожного експерименту відновлюється однакова тестова вибірка.

```
noise_steps = [24, 49, 99, 149]

results = []

test_images = x_test[:20]

for step in noise_steps:

    noisy, restored = restore_images(
        test_images,
        noise_step=step
    )

    metrics = evaluate_restoration(
        test_images, restored
    )

    results.append({
        "step": step + 1,
        **metrics
    })

    print(
        f"t={step + 1}: "
        f"MSE={metrics['MSE']:.4f}, "
        f"PSNR={metrics['PSNR']:.2f}, "
        f"SSIM={metrics['SSIM']:.4f}"
    )
```

Побудова графіка залежності якості відновлення від рівня шуму

```
steps = [r["step"] for r in results]
mse_values = [r["MSE"] for r in results]
psnr_values = [r["PSNR"] for r in results]
ssim_values = [r["SSIM"] for r in results]

fig, axes = plt.subplots(1, 3, figsize=(14, 4))

axes[0].plot(steps, mse_values, marker="o")
axes[0].set_title("MSE")

axes[1].plot(steps, psnr_values, marker="o")
axes[1].set_title("PSNR")

axes[2].plot(steps, ssim_values, marker="o")
axes[2].set_title("SSIM")

for ax in axes:
    ax.set_xlabel("Крок зашумлення")
    ax.grid(True)

plt.tight_layout()
plt.show()
```

Аналіз результатів. За результатами обчислень необхідно визначити, як рівень зашумлення впливає на точність відновлення зображень. Зазвичай збільшення рівня шуму ускладнює відновлення, однак фактичні значення метрик залежать від якості навчання мережі, вибраного розкладу шуму та випадкових компонентів зворотної дифузії.

Для достовірнішого порівняння бажано використовувати більшу тестову вибірку й усереднювати результати кількох запусків.

11. Аналіз отриманих результатів

У розглянутому прикладі реалізовано два взаємопов'язані процеси: пряме зашумлення зображень та зворотну дифузію за допомогою навченої U-Net.

Характеристика	Прямий процес	Зворотний процес
Призначення	Додавання шуму	Формування або відновлення зображення
Початковий стан	Реальне зображення	Шум або зашумлене зображення
Основні параметри	β_t, α_t, $\bar{\alpha_t}$	Прогноз шуму U-Net та дисперсія переходу
Навчувана мережа	Не потрібна	U-Net
Напрямок	$x_0 \rightarrow x_t$	$x_t \rightarrow x_0$
Результат	Зашумлене представлення	Згенероване або відновлене зображення

Під час аналізу експериментів слід оцінити збіжність функції втрат U-Net, візуальну якість генерації, точність відновлення зображень та чутливість отриманих результатів до кількості кроків і розкладу шуму.

Примітка. Наведений код є навчальним прикладом спрощеної DDPM. Візуальні ілюстрації в цьому розділі показують принципи роботи моделей, а не результати фактичного виконання наведеного коду. Кількісні результати, графіки навчання та згенеровані зображення необхідно отримати під час запуску програми.

Хід роботи

1. Виконати моделювання прямого та зворотного процесів дифузії для генерування й відновлення зображень за допомогою моделі DDPM (Denoising Diffusion Probabilistic Model) із використанням TensorFlow/Keras.
2. Завантажити набір даних Fashion-MNIST та підготувати зображення до моделювання:

- нормалізувати значення пікселів до діапазону ;
- перетворити зображення у тензори розмірності ;
- сформувати навчальну та тестову вибірки;
- візуалізувати декілька початкових зображень.

3. Реалізувати прямий процес дифузії (Forward Diffusion Process):

- задати загальну кількість кроків дифузії ;
- реалізувати лінійний розклад шуму (Linear Noise Schedule);
- обчислити коефіцієнти , та ;
- реалізувати формування зашумленого зображення за формулою:

4. Дослідити прямий процес дифузії:

- побудувати графіки зміни та ;
- сформувати зашумлені версії одного зображення для декількох кроків дифузії;
- візуалізувати послідовність зашумлення;
- проаналізувати зміну структури зображення зі збільшенням рівня шуму.

5. Побудувати спрощену нейронну мережу U-Net для прогнозування шуму, що включає:

- кодувальну частину (Encoder) зі згортковими шарами та зменшенням просторової розмірності;
- проміжний блок (Bottleneck);
- декодувальну частину (Decoder) зі збільшенням просторової розмірності;
- пропускні з’єднання (Skip Connections);
- синусоїдальне часове вбудовування (Time Embedding) для врахування номера кроку дифузії;
- вихідний згортковий шар для прогнозування шуму.

6. Проаналізувати архітектуру побудованої U-Net за допомогою model.summary(). Визначити кількість навчуваних параметрів, розмірності проміжних карт ознак і відповідність розмірності виходу розмірності вхідного зображення.
7. Реалізувати навчання дифузійної моделі:

- сформувати навчальні приклади шляхом додавання випадкового гауссового шуму на
- випадково вибраних кроках дифузії;
- використати MSE як функцію втрат між фактичним і прогнозованим шумом;
- застосувати оптимізатор Adam;
- реалізувати оновлення параметрів U-Net за допомогою ;
- виконати навчання моделі та зберегти її ваги.

8. Проаналізувати процес навчання:

- побудувати графік зміни MSE за епохами;
- визначити кінцеве значення функції втрат;
- оцінити характер збіжності моделі.

9. Реалізувати зворотний процес дифузії (Reverse Diffusion Process) із використанням навченої U-

Net:

- реалізувати прогнозування шуму ;
- обчислювати параметри зворотного переходу;
- послідовно формувати менш зашумлені зображення.

10. Виконати генерування нових зображень:

- сформувати початкові випадкові зображення ;
- виконати зворотний процес дифузії;
- згенерувати не менше 10 зображень;
- зберегти та візуалізувати проміжні етапи генерації;
- оцінити чіткість, різноманітність і наявність спотворень у згенерованих зображеннях.

11. Виконати відновлення зашумлених зображень:

- вибрати зображення з тестової вибірки;
- сформувати їхні зашумлені версії для кроків ;
- виконати зворотну дифузію для кожного рівня зашумлення;
- візуалізувати початкові, зашумлені та відновлені зображення.

12. Оцінити якість відновлення:

- обчислити MSE, PSNR та SSIM;

- побудувати графіки залежності метрик від рівня зашумлення;
- порівняти точність відновлення для різних кроків дифузії;
- визначити рівні шуму, за яких модель забезпечує найкращі та найгірші результати.

13. Виконати індивідуальне дослідження відповідно до номера варіанта. Для досліджуваного параметра використати не менше трьох значень або конфігурацій, якщо інше не передбачено умовою варіанта. Для експериментів, які змінюють архітектуру U-Net, кількість кроків дифузії або розклад шуму під час навчання, необхідно навчити окремі моделі.
14. Порівняти результати базової моделі та досліджуваних конфігурацій. Результати подати у вигляді таблиць, графіків та прикладів зображень. За можливості оцінити час навчання й генерації.
15. Сформулювати висновки щодо роботи прямого та зворотного процесів дифузії, точності прогнозування шуму, якості генерації й відновлення зображень та впливу параметрів, досліджених в індивідуальному завданні.

Рекомендовані налаштування за замовчуванням

Компонент	**Параметр**	**Значення**
Дані	Набір	Fashion-MNIST
	Розмір зображення	28×28×1
	Нормалізація	[-1,1]
Дифузія	Кількість кроків T	200
	Розклад шуму	Linear
		0.0001
		0.02
U-Net	Кількість рівнів Encoder	2
	Кількість рівнів Decoder	2
	Кількість каналів	32, 64, 128
	Ядро згортки	3×3
	Функція активації	SiLU
	Time Embedding	64
Навчання	Оптимізатор	Adam
	Learning rate	0.001
	Batch size	64
	Кількість епох	20
	Функція втрат	MSE
Оцінювання	Метрики відновлення	MSE, PSNR, SSIM
	Генерування	10 зображень
	Кроки зашумлення для відновлення	25, 50, 100, 150

Налаштування є початковими для навчального експерименту. У разі недостатньої якості генерації допускається збільшення кількості епох та коригування розкладу шуму з обов’язковим документуванням змін.

Індивідуальні завдання

Усі студенти виконують базові етапи лабораторної роботи: реалізацію прямого процесу дифузії, побудову та навчання спрощеної U-Net, генерування нових зображень і відновлення

зашумлених зображень. Відповідно до номера варіанта додатково необхідно виконати окреме експериментальне дослідження та порівняти отримані результати.

Для досліджуваного параметра необхідно використати не менше трьох значень або конфігурацій, якщо інше не передбачено умовою варіанта. Параметри, які не досліджуються, повинні залишатися незмінними. Якщо змінюються архітектура нейронної мережі, розклад шуму або параметри прямого дифузійного процесу, необхідно виконати окреме навчання моделі для кожної конфігурації.

Для порівняння точності відновлення використовувати однакові тестові зображення, а результати оцінювати за MSE, PSNR та SSIM. Якість безумовної генерації оцінювати візуально за чіткістю, різноманітністю зображень і наявністю артефактів.

№	Завдання	Що змінюється	Дані	Що необхідно порівняти
1	Дослідити вплив кількості кроків дифузії	T = 100, 200, 400	Fashion-MNIST	Якість генерації та відновлення, час генерації
2	Порівняти розклади шуму	Linear, Cosine; Linear зі зміненим β_{max}	Fashion-MNIST	Графіки β_t, $\bar{\alpha}_t$, MSE, PSNR, SSIM та якість генерації
3	Дослідити вплив максимального значення дисперсії шуму	β_{max} = 0.01, 0.02, 0.04	Fashion-MNIST	Накопичений рівень шуму, якість генерації та відновлення
4	Дослідити вплив початкового значення дисперсії шуму	β_{min} = 0.0001, 0.001, 0.005	Fashion-MNIST	Зміна $\bar{\alpha}_t$, збіжність MSE, якість відновлення
5	Дослідити вплив рівня зашумлення на відновлення	t = 25, 50, 100, 150	Fashion-MNIST	MSE, PSNR, SSIM та візуальна якість відновлення
6	Дослідити вплив кількості каналів U-Net	16–32–64, 32–64–128, 64–128–256	Fashion-MNIST	MSE, кількість параметрів, час навчання та якість генерації
7	Дослідити вплив глибини U-Net	1, 2, 3 рівні Encoder/Decoder	Fashion-MNIST	Точність прогнозування шуму, якість відновлення та обчислювальні витрати
8	Порівняти функції активації у U-Net	ReLU, LeakyReLU, SiLU	Fashion-MNIST	Збіжність MSE, стабільність навчання та якість генерації
9	Дослідити вплив розмірності Time Embedding	32, 64, 128	Fashion-MNIST	MSE, якість відновлення та кількість параметрів
10	Дослідити вплив Skip Connections	Без з'єднань, стандартні, з додатковими Residual Connections	Fashion-MNIST	MSE, збереження контурів та якість відновлення
11	Дослідити вплив learning rate	0.0001, 0.0005, 0.001	Fashion-MNIST	Швидкість збіжності MSE, стабільність навчання та якість генерації
12	Дослідити вплив batch size	32, 64, 128	Fashion-MNIST	Час навчання, збіжність MSE та якість результатів
13	Дослідити вплив кількості епох навчання	10, 20, 40 епох	Fashion-MNIST	MSE, якість генерації та відновлення, час навчання
14	Порівняти оптимізатори для навчання U-Net	Adam, AdamW, RMSprop	Fashion-MNIST	Збіжність функції втрат, стабільність та якість відновлення
15	Дослідити вплив кількості навчальних зображень	5 000, 15 000, 30 000	Fashion-MNIST	Якість генерації, MSE та залежність результатів від обсягу даних
16	Порівняти способи семплювання	DDPM та DDIM із 50 і 100 кроками	Fashion-MNIST	Час генерації, чіткість і різноманітність зображень
17	Дослідити відновлення зображень із різною початковою складністю	Зображення 3 класів одягу	Fashion-MNIST	MSE, PSNR, SSIM та особливості відновлення різних об'єктів
18	Дослідити вплив випадкової ініціалізації	3 різні значення random seed	Fashion-MNIST	Стабільність навчання, варіативність генерації та метрики відновлення
19	Дослідити ефективність DDPM на іншому наборі зображень	MNIST замість Fashion-MNIST; однакові параметри	MNIST, Fashion-MNIST	Збіжність MSE, якість генерації та відновлення для двох наборів
20	Виконати комплексне порівняння моделей із різними розкладами шуму	Linear, Cosine, T = 100 і 200	Fashion-MNIST	Якість генерації, MSE, PSNR, SSIM, час навчання та генерації

Варіанти 1–4. Під час дослідження параметрів прямого процесу дифузії необхідно будувати графіки β_t та $\bar{\alpha}_t$. Для кожної конфігурації слід навчити відповідну модель, оскільки зміна розкладу шуму змінює розподіл навчальних прикладів. Під час порівняння кількості кроків потрібно враховувати різницю кінцевих рівнів шуму.

Варіант 7. Під час зміни глибини U-Net необхідно забезпечити відповідність просторових розмірностей карт ознак у кодувальній та декодувальній частинах. Для трьох рівнів зменшення розмірності зображень 28×28 може знадобитися додаткове узгодження розмірів перед конкатенацією.

Варіант 10. Порівняти U-Net без пропускних з'єднань, базову U-Net зі Skip Connections і модифіковану архітектуру з додатковими залишковими з'єднаннями. В усіх конфігураціях вихід мережі повинен зберігати розмірність 28×28×1.

Варіант 13. Доцільно використати одну модель і зберігати її стан після 10, 20 та 40 епох. Це дозволить порівнювати результати на різних етапах одного процесу навчання без повторного запуску.

Варіант 16. Для порівняння DDPM і DDIM необхідно додатково реалізувати алгоритм DDIM sampling. Слід використовувати одну й ту саму навчену U-Net та узгоджений розклад шуму. Це дозволить дослідити прискорення генерації без повторного навчання моделі.

Варіант 17. Рекомендується вибрати три класи, наприклад футболки, сумки та взуття. Для кожного класу використати однакову кількість тестових зображень і однаковий рівень зашумлення. Оцінити середні значення MSE, PSNR і SSIM окремо для кожного класу.

Варіант 18. Під час дослідження впливу необхідно розрізняти випадковість навчання та випадковість генерації. Для аналізу стабільності навчання слід виконати три незалежні навчання моделі з різними значеннями seed. Для аналізу генерації можна використовувати одну навчену модель із різними початковими шумовими тензорами.

Варіант 19. Для коректного порівняння MNIST і Fashion-MNIST необхідно використовувати однакову архітектуру, розклад шуму, кількість епох та інші параметри навчання. Моделі навчаються окремо для кожного набору даних.

Варіант 20. Порівняти чотири конфігурації: Linear із T = 100, Linear із T = 200, Cosine із T = 100 та Cosine із T = 200. Для кожної конфігурації необхідно навчити окрему модель та виконати однакові процедури оцінювання.

Вимоги до результатів індивідуального дослідження

Результати виконання індивідуального завдання повинні містити:

1. Опис досліджуваного параметра та перелік використаних конфігурацій.
2. Результати навчання моделей, зокрема значення MSE та графіки функції втрат.
3. Приклади згенерованих або відновлених зображень для кожної конфігурації.
4. Таблицю порівняння результатів, що включає відповідні метрики та, за необхідності, час навчання або генерації.
5. Аналіз впливу досліджуваного параметра на якість результатів та сформульовані висновки.

Для оцінювання відновлення використовувати однакові тестові зображення. У випадку стохастичного семплювання бажано виконувати кілька повторень та усереднювати результати.

Примітка. Варіанти 6, 7, 10, 16 і 20 передбачають суттєвіші модифікації базового програмного прикладу. Для виконання цих завдань студенти повинні самостійно адаптувати архітектуру моделі або алгоритм семплювання. Варіанти з повторним навчанням кількох моделей потребують більше обчислювального часу, тому рекомендується використовувати Google Colab із GPU.

Зміст звіту

Звіт з лабораторної роботи повинен містити:

1. Титульний аркуш із зазначенням назви лабораторної роботи, номера варіанта, прізвища

   студента та навчальної групи.
2. Мету роботи.
3. Опис набору даних та його підготовки:

- назву набору даних, кількість і розмірність зображень;
- спосіб нормалізації;
- приклади початкових зображень.

4. Опис прямого процесу дифузії:

- кількість кроків дифузії та параметри розкладу шуму;
- графіки зміни та ;
- приклади зображень на різних кроках зашумлення.

5. Опис архітектури та навчання U-Net:

- структуру моделі ( ), основні шари та кількість параметрів;
- спосіб врахування номера кроку дифузії (Time Embedding);
- параметри навчання: оптимізатор, learning rate, batch size, кількість епох і функцію втрат;
- графік зміни MSE та кінцеве значення функції втрат.

6. Результати генерування зображень:

- приклади не менше 10 згенерованих зображень;
- візуалізацію проміжних етапів зворотної дифузії;
- короткий аналіз якості сформованих зображень.

7. Результати відновлення зашумлених зображень:

- порівняння початкових, зашумлених і відновлених зображень;
- таблицю значень MSE, PSNR та SSIM для різних рівнів зашумлення;
- графіки залежності метрик від кроку зашумлення;
- короткий аналіз точності відновлення.

8. Результати виконання індивідуального завдання:

- умову варіанта та перелік досліджених параметрів;
- таблицю порівняння отриманих результатів;
- відповідні графіки та приклади зображень;
- аналіз впливу досліджуваного параметра на якість генерації або відновлення.

9. Програмний код, що включає підготовку даних, реалізацію прямого процесу дифузії, побудову та навчання U-Net, зворотну дифузію, генерування й відновлення зображень, обчислення метрик і виконання індивідуального завдання.
10. Висновки, у яких необхідно стисло зазначити:

- особливості прямого та зворотного процесів дифузії;
- результати навчання U-Net для прогнозування шуму;
- якість генерування нових зображень;
- точність відновлення зашумлених зображень;

- вплив рівня зашумлення та параметра, дослідженого в індивідуальному завданні.

Контрольні запитання

1. Що таке генеративна дифузійна модель і які завдання вона розв’язує?
2. У чому полягає основний принцип роботи Denoising Diffusion Probabilistic Model (DDPM)?
3. Чим прямий процес дифузії (Forward Diffusion Process) відрізняється від зворотного (Reverse Diffusion Process)?
4. Чому прямий процес дифузії описується марковським ланцюгом?
5. Яку роль відіграє гауссовий шум у дифузійній моделі?
6. Що визначає параметр β_t і як його значення впливає на зашумлення зображення?
7. Що характеризують коефіцієнти α_t та $\bar{\alpha}_t$?
8. Як за формулою прямої дифузії отримати зашумлене зображення x_t без послідовного виконання всіх попередніх кроків?
9. Що таке Noise Schedule і для чого він використовується в дифузійній моделі?
10. Чим відрізняються лінійний (Linear) та косинусний (Cosine) розклади шуму?
11. Як кількість кроків дифузії T впливає на процес зашумлення та генерації зображень?
12. Що характеризує відношення сигналу до шуму (SNR) і як воно змінюється під час прямої дифузії?
13. Чому за достатньо тривалого та належно налаштованого прямого процесу розподіл зашумлених зображень наближається до стандартного нормального розподілу?
14. У чому полягає завдання нейронної мережі під час навчання DDPM?
15. Чому для прогнозування шуму в дифузійних моделях використовується архітектура U-Net?
16. Які функції виконують Encoder, Bottleneck та Decoder в архітектурі U-Net?
17. Для чого в U-Net використовуються пропускні з’єднання (Skip Connections)?
18. Яку роль виконує часове вбудовування (Time Embedding) і чому нейронній мережі необхідно знати номер кроку дифузії?
19. Чому вихідний тензор U-Net повинен мати таку саму розмірність, як вхідне зображення?
20. Яка функція втрат використовується для навчання спрощеної DDPM і чому?
21. Як формуються вхідні дані та цільові значення для навчання U-Net?
22. Для чого під час навчання випадково вибирають номери кроків дифузії?
23. Яку роль виконує tf.GradientTape під час навчання дифузійної моделі в TensorFlow?
24. Як здійснюється один крок зворотної дифузії з використанням прогнозованого нейронною мережею шуму?
25. Чому в зворотному процесі DDPM додається випадковий шум і чому на останньому кроці його не додають?
26. Як здійснюється генерування нового зображення з випадкового гауссового шуму?
27. Чим генерування нового зображення відрізняється від відновлення зашумленого зображення?
28. Як рівень початкового зашумлення впливає на точність відновлення зображення?

29. Для чого використовуються метрики MSE, PSNR та SSIM і як інтерпретуються їхні значення?
30. Чому MSE, PSNR та SSIM не є достатніми для оцінювання якості безумовної генерації нових зображень?
31. Як кількість кроків зворотного процесу впливає на обчислювальні витрати та якість генерації?
32. Чим відрізняються алгоритми семплювання DDPM та DDIM?
33. Чому зміна розкладу шуму зазвичай потребує повторного навчання дифузійної моделі?
34. Як можна оцінити якість навчання U-Net за графіком функції втрат та згенерованими зображеннями?
35. Які основні переваги та обмеження дифузійних моделей порівняно з іншими генеративними підходами, зокрема GAN і VAE?